

CHAPTER 5 — SYSTEM IMPLEMENTATION

4.1 Introduction

This chapter presents the actual implementation of the RakshaSafe system through its real source code. Every listing below is the complete, unmodified file from the project repository, syntax-highlighted for readability: keywords in blue, strings in maroon, comments in green italic, types and classes in teal, and numbers in green. The listings follow the runtime layers of the system — backend foundation, data models, API routes, controllers and services, the Python AI service, and the React frontend — and together they form a complete, buildable implementation that matches the design of Chapter 3.

Code is shown in Consolas with a light shaded background. Each listing states the repository path and the line count so that the printed listing can be checked against the electronic source.

4.2 Backend — entry, configuration and middleware

Listing 4.1 — backend/src/server.ts

Application entry point: connects the database and starts the HTTP server.

```
import { createApp } from './app.js'
import { connectDatabase, disconnectDatabase } from './config/db.js'
import { env } from './config/env.js'

const app = createApp()

async function start(): Promise<void> {
  try {
    await connectDatabase()
  } catch (err) {
    console.warn(`[api] MongoDB unreachable at startup: ${err as Error}.message`)
    console.warn('[api] Starting API anyway; /api/health will report db: connected=false')
  }

  const server = app.listen(env.port, () => {
    console.log(`[api] RakshaSafe backend listening on http://localhost:${env.port}`)
  })

  const shutdown = async (signal: string) => {
    console.log(`[api] ${signal} received, shutting down...`)
    server.close(async () => {
      await disconnectDatabase()
      process.exit(0)
    })
  }

  process.on('SIGINT', () => void shutdown('SIGINT'))
  process.on('SIGTERM', () => void shutdown('SIGTERM'))
}

start().catch((err) => {
  console.error('[api] Failed to start:', err)
  process.exit(1)
})
```

Listing 4.2 — backend/src/app.ts

Express application wiring: middleware, routes and error handling.

```
import cors from 'cors'
import express from 'express'
import { env } from './config/env.js'
import { notFound, errorHandler } from './middleware/error.js'
import routes from './routes/index.js'
```

```

export function createApp() {
  const app = express()

  // Do not fingerprint the framework in responses.
  app.disable('x-powered-by')

  app.use(
    cors({
      origin: env.corsOrigins,
      credentials: true,
    }),
  )
  app.use(express.json({ limit: '1mb' }))
  app.use(express.urlencoded({ extended: true }))

  app.get('/', (_req, res) => {
    res.json({
      service: 'RakshaSafe API',
      version: '0.1.0',
      docs: '/api/health',
    })
  })

  app.use('/api', routes)

  app.use(notFound)
  app.use(errorHandler)

  return app
}

```

Listing 4.3 — backend/src/routes/index.ts

Central API router mounting every feature module.

```

import { Router } from 'express'
import mongoose from 'mongoose'
import authRoutes from './auth.js'
import adminRoutes from './admin.js'
import emergencyContactRoutes from './emergencyContacts.js'
import facilityRoutes from './facilities.js'
import incidentRoutes from './incidents.js'
import nearbyRoutes from './nearby.js'
import notificationRoutes from './notifications.js'
import osmRoutes from './osm.js'
import weatherRoutes from './weather.js'
import rescueTeamRoutes from './rescueTeams.js'
import responderRoutes from './responder.js'
import unsafeReportRoutes from './unsafeReports.js'
import geocodeRoutes from './geocode.js'

const router = Router()

router.use('/auth', authRoutes)
router.use('/admin', adminRoutes)
router.use('/emergency-contacts', emergencyContactRoutes)
router.use('/incidents', incidentRoutes)
router.use('/facilities', facilityRoutes)
router.use('/nearby-resources', nearbyRoutes)
router.use('/nearby', osmRoutes)
router.use('/weather', weatherRoutes)
router.use('/notifications', notificationRoutes)
router.use('/rescue-teams', rescueTeamRoutes)
router.use('/responder', responderRoutes)
router.use('/unsafe-reports', unsafeReportRoutes)
router.use('/geocode', geocodeRoutes)

router.get('/health', (_req, res) => {
  const dbState = mongoose.connection.readyState
  res.json({
    success: true,
  })
})

```

```

    service: 'rakshasafe-backend',
    status: 'ok',
    db: {
      state: dbState, // 0 disconnected, 1 connected, 2 connecting, 3 disconnecting
      connected: dbState === 1,
    },
    time: new Date().toISOString(),
  })
})

export default router

```

Listing 4.4 — backend/src/config/env.ts

Typed environment configuration with safe development defaults.

```

import 'dotenv/config'

function required(name: string, fallback: string): string {
  return process.env[name] ?? fallback
}

/** Refuse to boot in production with a placeholder or missing secret key. */
function requiredSecret(name: string, devFallback: string): string {
  const value = process.env[name]
  if (value !== undefined && value.trim() !== '' && value !== devFallback) {
    return value
  }
  const isProd = process.env.NODE_ENV === 'production'
  if (isProd) {
    throw new Error(
      `Missing ${name}. Set a real value in the environment (NODE_ENV=production refuses the development fallback).`,
    )
  }
  return value && value.trim() !== '' ? value : devFallback
}

function positiveNumber(name: string, fallback: number): number {
  const raw = process.env[name]
  if (raw === undefined || raw.trim() === '') return fallback
  const parsed = Number(raw.trim())
  if (!Number.isFinite(parsed) || parsed <= 0) return fallback
  return parsed
}

const isProduction = process.env.NODE_ENV === 'production'

export const env = {
  nodeEnv: required('NODE_ENV', 'development'),
  port: Number(required('PORT', '5000')),
  mongoUri: isProduction
    ? requiredSecret('MONGO_URI', '')
    : required('MONGO_URI', 'mongodb://127.0.0.1:27017/rakshasafe'),
  jwtSecret: requiredSecret('JWT_SECRET', 'rakshasafe-dev-secret-change-me'),
  jwtExpiresIn: required('JWT_EXPIRES_IN', '7d'),
  aiServiceUrl: required('AI_SERVICE_URL', 'http://localhost:8000'),
  aiServiceTimeoutMs: Number(required('AI_SERVICE_TIMEOUT_MS', '10000')),
  /**
   * CORS allowlist: comma-separated frontend origins. The dev server (5173)
   * and `vite preview` (4173) are both allowed by default so admin pages
   * never fail with a network error just because the bundle is served from
   * the preview port. Production should set an explicit single origin.
   */
  corsOrigins: required('CORS_ORIGIN', 'http://localhost:5173,http://localhost:4173')
    .split(',')
    .map((o) => o.trim())
    .filter((o) => o !== ''),
  logLevel: required('LOG_LEVEL', 'dev'),
  /**
   * External nearby-places provider (Google Places API, New).
   * Server-side only – never exposed to the frontend. Disabled unless
   * explicitly enabled AND a key is present; the app is fully functional
   * on RakshaSafe database resources alone.
   */

```

```

    */
    googlePlacesApiKey: required('GOOGLE_PLACES_API_KEY', ''),
    nearbyExternalEnabled: required('NEARBY_EXTERNAL_ENABLED', 'false').trim().toLowerCase() === 'true',
    nearbyExternalTimeoutMs: positiveNumber('NEARBY_EXTERNAL_TIMEOUT_MS', 8000),
    nearbyRadiusKm: positiveNumber('NEARBY_RADIUS_KM', 25),
  } as const

```

Listing 4.5 — backend/src/config/db.ts

MongoDB connection with tolerant timeout and background reconnect.

```

import mongoose from 'mongoose'
import { env } from './env.js'

// Server selection must tolerate a waking cloud cluster. A free-tier Atlas
// cluster that was paused/idle can take several seconds to accept the first
// connection, so a tight timeout makes startup fail even though the cluster
// becomes reachable moments later.
const SERVER_SELECTION_TIMEOUT_MS = 10_000

// Bounded backoff for background reconnection attempts after an initial
// connection failure. The API stays up and `requireDb` keeps rejecting
// DB-dependent requests until one of these attempts succeeds, so readiness
// recovers without a manual restart.
const RETRY_BASE_DELAY_MS = 2_000
const RETRY_MAX_DELAY_MS = 15_000

let retryAttempt = 0
let retryTimer: NodeJS.Timeout | null = null

async function tryConnect(): Promise<boolean> {
  try {
    await mongoose.connect(env.mongoUri, { serverSelectionTimeoutMS: SERVER_SELECTION_TIMEOUT_MS })
    console.log(`[db] Connected to MongoDB (${mongoose.connection.name})`)
    retryAttempt = 0
    return true
  } catch (err) {
    console.warn(`[db] MongoDB connection attempt failed: ${err as Error}.message`)
    return false
  }
}

function scheduleRetry(): void {
  const delay = Math.min(RETRY_BASE_DELAY_MS * 2 ** retryAttempt, RETRY_MAX_DELAY_MS)
  retryAttempt += 1
  retryTimer = setTimeout(() => {
    void tryConnect().then((ok) => {
      if (!ok) scheduleRetry()
    })
  }, delay)
}

export async function connectDatabase(): Promise<void> {
  mongoose.connection.on('disconnected', () => {
    console.warn('[db] MongoDB connection lost')
  })
  mongoose.connection.on('connected', () => {
    console.log('[db] MongoDB connection established')
  })

  const ok = await tryConnect()
  if (!ok) {
    // Keep the original contract: the API starts while MongoDB is down and
    // `requireDb` gates DB-dependent routes. The Background retry loop will
    // flip readyState to 1 as soon as MongoDB is reachable again.
    console.warn('[db] Initial MongoDB connection failed; retrying in the background.')
    scheduleRetry()
    throw new Error('Initial MongoDB connection failed.')
  }
}

export async function disconnectDatabase(): Promise<void> {
  if (retryTimer) {

```

```

    clearTimeout(retryTimer)
    retryTimer = null
  }
  await mongoose.disconnect()
  console.log('[db] MongoDB connection closed')
}

```

Listing 4.6 — backend/src/middleware/auth.ts

JWT verification plus ADMIN and RESPONDER role guards.

```

import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { User, UserRole } from '../models/User.js'
import { forbidden, unauthorized } from '../utils/errors.js'
import { verifyAuthToken } from '../utils/jwt.js'

function extractBearerToken(req: Request): string | null {
  const header = req.headers.authorization
  if (!header) return null
  const [scheme, token] = header.split(' ')
  if (scheme?.toLowerCase() !== 'bearer' || !token) return null
  return token
}

/**
 * Require a valid JWT and Load the account's current state from the
 * database whenever MongoDB is reachable. Deleted or deactivated accounts
 * lose access immediately; role changes take effect on the next request
 * instead of lingering until the token expires. Attaches `{ userId, role }`
 * to `req.auth`, preferring DB state over token claims.
 *
 * When the database is unreachable the token is still accepted (the same
 * policy as the database-independent OSM/weather/geocode routes); routes
 * that need the database are independently rejected by `requireDb`.
 */
export async function requireAuth(req: Request, _res: Response, next: NextFunction): Promise<void> {
  const token = extractBearerToken(req)
  if (!token) {
    next(unauthorized('Authentication required. Provide a Bearer token.'))
    return
  }
  const payload = verifyAuthToken(token)
  if (!payload) {
    next(unauthorized('Invalid or expired token.'))
    return
  }

  if (mongoose.connection.readyState === 1) {
    try {
      const user = await User.findById(payload.sub).select('_id role isActive').lean()
      if (!user) {
        next(unauthorized('Account no longer exists. Please sign in again.'))
        return
      }
      if (user.isActive === false) {
        next(unauthorized('Account is deactivated. Contact support.'))
        return
      }
      req.auth = { userId: String(user._id), role: user.role }
      next()
      return
    } catch {
      // DB glitch mid-request: fall back to token-only auth for this call.
    }
  }

  req.auth = { userId: payload.sub, role: payload.role }
  next()
}

/** Require the authenticated user to have the ADMIN role. Use after requireAuth. */
export function requireAdmin(req: Request, _res: Response, next: NextFunction): void {
  if (!req.auth) {
    next(unauthorized('Authentication required.'))
  }
}

```

```

    return
  }
  if (req.auth.role !== UserRole.ADMIN) {
    next(forbidden('Administrator access required.'))
    return
  }
  next()
}

/** Require the authenticated user to have the RESPONDER role. Use after requireAuth. */
export function requireResponder(req: Request, _res: Response, next: NextFunction): void {
  if (!req.auth) {
    next(unauthorized('Authentication required.'))
    return
  }
  if (req.auth.role !== UserRole.RESPONDER) {
    next(forbidden('Responder access required.'))
    return
  }
  next()
}

```

Listing 4.7 — backend/src/middleware/requireDb.ts

Gate that returns 503 while the database is unreachable.

```

import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { serviceUnavailable } from '../utils/errors.js'

/**
 * Reject requests that need the database while MongoDB is unreachable,
 * instead of letting them hang until a timeout.
 */
export function requireDb(_req: Request, _res: Response, next: NextFunction): void {
  if (mongoose.connection.readyState !== 1) {
    next(serviceUnavailable('Database unavailable. Please try again later.'))
    return
  }
  next()
}

```

Listing 4.8 — backend/src/middleware/error.ts

Central error-to-HTTP-response translator.

```

import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import { HttpError } from '../utils/errors.js'

export function notFound(req: Request, res: Response): void {
  res.status(404).json({
    success: false,
    error: 'Not found',
    path: req.originalUrl,
  })
}

function isDuplicateKeyError(err: unknown): err is { code: number; keyValue?: Record<string, unknown> } {
  return (
    typeof err === 'object' &&
    err !== null &&
    'code' in err &&
    (err as { code: unknown }).code === 11000
  )
}

function isJsonParseError(err: unknown): boolean {
  return (
    typeof err === 'object' &&
    err !== null &&
    (err as { type?: unknown }).type === 'entity.parse.failed' &&
    (err as { status?: unknown }).status === 400
  )
}

```

```

function isPayloadTooLarge(err: unknown): boolean {
  return (
    typeof err === 'object' &&
    err !== null &&
    (err as { type?: unknown }).type === 'entity.too.large'
  )
}

export function errorHandler(
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction,
): void {
  if (err instanceof HttpError) {
    res.status(err.statusCode).json({
      success: false,
      error: err.message,
      ...(err.details !== undefined ? { details: err.details } : {}),
    })
    return
  }

  if (isDuplicateKeyError(err)) {
    const fields = Object.keys(err.keyValue ?? {})
    res.status(409).json({
      success: false,
      error:
        fields.length > 0
          ? `An account with this ${fields.join(' and ')} already exists.`
          : 'A duplicate record already exists.',
    })
    return
  }

  if (err instanceof mongoose.Error.ValidationError) {
    res.status(400).json({
      success: false,
      error: 'Validation failed.',
      details: Object.values(err.errors).map((e) => ({
        field: (e as { path?: string }).path ?? 'unknown',
        message: e.message,
      })),
    })
    return
  }

  if (isJsonParseError(err)) {
    res.status(400).json({
      success: false,
      error: 'Invalid JSON in request body.',
    })
    return
  }

  if (isPayloadTooLarge(err)) {
    res.status(413).json({
      success: false,
      error: 'Request payload too large.',
    })
    return
  }

  console.error('[error]', err)
  res.status(500).json({
    success: false,
    error: 'Internal server error',
  })
}

```

Listing 4.9 — backend/src/middleware/rateLimit.ts

In-memory per-IP rate limiting for sensitive endpoints.

```

import type { NextFunction, Request, Response } from 'express'

interface RateLimitOptions {
  windowMs: number
  max: number
  message?: string
}

interface Bucket {
  count: number
  resetAt: number
}

const buckets = new Map<string, Bucket>()

function keyFor(req: Request): string {
  return `${req.ip ?? 'unknown'}:${req.originalUrl ?? req.path}`
}

/** Exponential backoff on failure keeps the map from growing unboundedly. */
function reap(now: number): void {
  if (buckets.size < 10_000) return
  for (const [key, bucket] of buckets) {
    if (bucket.resetAt <= now) buckets.delete(key)
  }
}

/**
 * Lightweight in-memory fixed-window rate limiter keyed by client IP and
 * route. Suitable for a single-instance deploy; no external dependency.
 */
export function rateLimit({ windowMs, max, message }: RateLimitOptions) {
  return (req: Request, res: Response, next: NextFunction): void => {
    const now = Date.now()
    reap(now)
    const key = keyFor(req)
    const bucket = buckets.get(key)
    if (!bucket || bucket.resetAt <= now) {
      buckets.set(key, { count: 1, resetAt: now + windowMs })
      next()
      return
    }
    bucket.count += 1
    if (bucket.count > max) {
      res.status(429).json({
        success: false,
        error: message ?? 'Too many requests. Please try again shortly.',
      })
      return
    }
    next()
  }
}

```

Listing 4.10 — backend/src/utils/jwt.ts

Signing and verification of authentication tokens.

```

import jwt from 'jsonwebtoken'
import { env } from '../config/env.js'
import { UserRole } from '../models/User.js'

export interface AuthTokenPayload {
  sub: string
  role: UserRole
}

/** Sign a JWT for an authenticated user. Payload carries only id + role. */
export function signAuthToken(userId: string, role: UserRole): string {
  const payload: AuthTokenPayload = { sub: userId, role }
  return jwt.sign(payload, env.jwtSecret, {
    expiresIn: env.jwtExpiresIn as jwt.SignOptions['expiresIn'],
  })
}

```

```

}

/** Verify a JWT and return its payload, or null when invalid/expired. */
export function verifyAuthToken(token: string): AuthTokenPayload | null {
  try {
    const decoded = jwt.verify(token, env.jwtSecret)
    if (typeof decoded === 'string') return null
    const { sub, role } = decoded as Partial<AuthTokenPayload>
    const allowedRoles = Object.values(UserRole) as string[]
    if (typeof sub !== 'string' || typeof role !== 'string' || !allowedRoles.includes(role)) {
      return null
    }
    return { sub, role: role as UserRole }
  } catch {
    return null
  }
}

```

Listing 4.11 — backend/src/utils/errors.ts

Typed HTTP error helpers used across controllers.

```

/** HTTP-aware error used by controllers and middleware. */
export class HttpError extends Error {
  statusCode: number
  details?: unknown

  constructor(statusCode: number, message: string, details?: unknown) {
    super(message)
    this.name = 'HttpError'
    this.statusCode = statusCode
    this.details = details
  }
}

export function badRequest(message: string, details?: unknown): HttpError {
  return new HttpError(400, message, details)
}

export function unauthorized(message = 'Authentication required'): HttpError {
  return new HttpError(401, message)
}

export function forbidden(message = 'Access denied'): HttpError {
  return new HttpError(403, message)
}

export function notFoundError(message = 'Not found'): HttpError {
  return new HttpError(404, message)
}

export function conflict(message: string): HttpError {
  return new HttpError(409, message)
}

export function serviceUnavailable(message: string): HttpError {
  return new HttpError(503, message)
}

```

Listing 4.12 — backend/src/utils/password.ts

bcrypt password hashing and comparison.

```

import bcrypt from 'bcryptjs'

const SALT_ROUNDS = 12

/** Hash a plain-text password. Never store or return plain-text passwords. */
export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, SALT_ROUNDS)
}

```

```

/** Compare a plain-text password against a stored hash. */
export async function comparePassword(password: string, passwordHash: string): Promise<boolean> {
  return bcrypt.compare(password, passwordHash)
}

```

4.3 Backend — data models (Mongoose schemas)

Listing 4.13 — backend/src/models/User.ts

User accounts with USER, ADMIN and RESPONDER roles.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum UserRole {
  USER = 'USER',
  ADMIN = 'ADMIN',
  RESPONDER = 'RESPONDER',
}

export interface IUser extends Document {
  _id: Types.ObjectId
  name: string
  email: string
  phone: string
  passwordHash: string
  role: UserRole
  language?: string
  isActive: boolean
  createdAt: Date
  updatedAt: Date
}

const UserSchema = new Schema<IUser>(
  {
    name: { type: String, required: true, trim: true },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
      index: true,
    },
    phone: {
      type: String,
      required: true,
      unique: true,
      trim: true,
      index: true,
    },
    passwordHash: { type: String, required: true },
    role: { type: String, enum: UserRole, required: true, default: UserRole.USER },
    language: { type: String, trim: true },
    isActive: { type: Boolean, required: true, default: true },
  },
  { timestamps: true },
)

export const User = mongoose.model<IUser>('Users', UserSchema)

```

Listing 4.14 — backend/src/models/Incident.ts

SOS and emergency incidents with the 7-state lifecycle.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum IncidentType {
  SAFETY = 'Safety',
  DISASTER = 'Disaster',
}

export enum IncidentPriority {
  LOW = 'LOW',

```

```

MEDIUM = 'MEDIUM',
HIGH = 'HIGH',
CRITICAL = 'CRITICAL',
}

export enum IncidentStatus {
  REPORTED = 'REPORTED',
  ACKNOWLEDGED = 'ACKNOWLEDGED',
  ASSIGNED = 'ASSIGNED',
  IN_PROGRESS = 'IN_PROGRESS',
  RESOLVED = 'RESOLVED',
  CLOSED = 'CLOSED',
  CANCELLED = 'CANCELLED',
}

export interface IIncident extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  type: IncidentType
  category: string
  description: string
  priority: IncidentPriority
  status: IncidentStatus
  locationId?: Types.ObjectId
  createdAt: Date
  updatedAt: Date
  resolvedAt?: Date
}

const IncidentSchema = new Schema<IIncident>(
{
  userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
  type: { type: String, enum: IncidentType, required: true },
  category: { type: String, required: true, trim: true },
  description: { type: String, required: true, trim: true },
  priority: { type: String, enum: IncidentPriority, required: true },
  status: {
    type: String,
    enum: IncidentStatus,
    required: true,
    default: IncidentStatus.REPORTED,
  },
  locationId: { type: Schema.Types.ObjectId, ref: 'Locations', index: true },
  resolvedAt: { type: Date },
},
{ timestamps: true },
)

IncidentSchema.index({ userId: 1, createdAt: -1 })
IncidentSchema.index({ status: 1, createdAt: -1 })
IncidentSchema.index({ priority: 1, createdAt: -1 })
IncidentSchema.index({ type: 1, status: 1 })

export const Incident = mongoose.model<IIncident>('Incidents', IncidentSchema)

```

Listing 4.15 — backend/src/models/Location.ts

Real geographic points with resolved address fields.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export interface ILocation extends Document {
  _id: Types.ObjectId
  latitude: number
  longitude: number
  address?: string
  city?: string
  /** Sub-locality resolved by reverse geocoding (never invented). */
  district?: string
  state?: string
  /** Postal code resolved by reverse geocoding (never invented). */
  postalCode?: string
  country?: string
  accuracy?: number
}

```

```

    createdAt: Date
    updatedAt: Date
  }

  const LocationSchema = new Schema<ILocation>(
    {
      latitude: { type: Number, required: true, min: -90, max: 90 },
      longitude: { type: Number, required: true, min: -180, max: 180 },
      address: { type: String, trim: true },
      city: { type: String, trim: true },
      district: { type: String, trim: true },
      state: { type: String, trim: true },
      postalCode: { type: String, trim: true },
      country: { type: String, trim: true },
      accuracy: { type: Number, min: 0 },
    },
    { timestamps: true },
  )

  LocationSchema.index({ latitude: 1, longitude: 1 })
  LocationSchema.index({ city: 1, state: 1, country: 1 })

  export const Location = mongoose.model<ILocation>('Locations', LocationSchema)

```

Listing 4.16 — backend/src/models/RiskAssessment.ts

Stored deterministic AI risk results keyed by location.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum RiskLevel {
  LOW = 'LOW',
  MEDIUM = 'MEDIUM',
  HIGH = 'HIGH',
  CRITICAL = 'CRITICAL',
}

export interface IRiskAssessment extends Document {
  _id: Types.ObjectId
  locationId: Types.ObjectId
  riskScore: number
  riskLevel: RiskLevel
  modelVersion: string
  inputFactors?: Record<string, unknown>[]
  assessedAt: Date
  createdAt: Date
}

const RiskAssessmentSchema = new Schema<IRiskAssessment>(
  {
    locationId: { type: Schema.Types.ObjectId, ref: 'Locations', required: true, index: true },
    riskScore: { type: Number, required: true, min: 0, max: 100 },
    riskLevel: { type: String, enum: RiskLevel, required: true },
    modelVersion: { type: String, required: true, trim: true },
    inputFactors: { type: [Schema.Types.Mixed], default: [] },
    assessedAt: { type: Date, required: true, default: Date.now },
  },
  { timestamps: true },
)

RiskAssessmentSchema.index({ locationId: 1, assessedAt: -1 })
RiskAssessmentSchema.index({ riskLevel: 1, assessedAt: -1 })

export const RiskAssessment = mongoose.model<IRiskAssessment>(
  'RiskAssessments',
  RiskAssessmentSchema,
)

```

Listing 4.17 — backend/src/models/EmergencyContact.ts

Trusted contacts with one primary per user.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

```

```

export interface IEmergencyContact extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  name: string
  phone: string
  email?: string
  relationship?: string
  notifyViaSms: boolean
  notifyViaEmail: boolean
  isPrimary: boolean
  createdAt: Date
  updatedAt: Date
}

const EmergencyContactSchema = new Schema<IEmergencyContact>({
  {
    userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
    name: { type: String, required: true, trim: true },
    phone: { type: String, required: true, trim: true },
    email: { type: String, lowercase: true, trim: true },
    relationship: { type: String, trim: true },
    notifyViaSms: { type: Boolean, required: true, default: false },
    notifyViaEmail: { type: Boolean, required: true, default: false },
    isPrimary: { type: Boolean, required: true, default: false },
  },
  { timestamps: true },
})

// Ensure only one primary contact per user
EmergencyContactSchema.index(
  { userId: 1, isPrimary: 1 },
  { unique: true, partialFilterExpression: { isPrimary: true } },
)

export const EmergencyContact = mongoose.model<IEmergencyContact>(
  'EmergencyContacts',
  EmergencyContactSchema,
)

```

Listing 4.18 — backend/src/models/Notification.ts

Event records with honest channel outcomes.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum NotificationChannel {
  EMAIL = 'Email',
  SMS = 'SMS',
  WHATSAPP = 'WhatsApp',
  IN_APP = 'In-App',
}

export enum NotificationStatus {
  QUEUED = 'QUEUED',
  SENT = 'SENT',
  DELIVERED = 'DELIVERED',
  FAILED = 'FAILED',
  NOT_CONFIGURED = 'NOT_CONFIGURED',
  UNAVAILABLE = 'UNAVAILABLE',
}

export interface INotification extends Document {
  _id: Types.ObjectId
  incidentId: Types.ObjectId
  contactId?: Types.ObjectId
  channel: NotificationChannel
  status: NotificationStatus
  providerResponse?: string
  attemptCount: number
  lastAttemptAt?: Date
  createdAt: Date
  updatedAt: Date
}

```

```

const NotificationSchema = new Schema<INotification>(
  {
    incidentId: { type: Schema.Types.ObjectId, ref: 'Incidents', required: true, index: true },
    contactId: { type: Schema.Types.ObjectId, ref: 'EmergencyContacts', index: true },
    channel: { type: String, enum: NotificationChannel, required: true },
    status: {
      type: String,
      enum: NotificationStatus,
      required: true,
      default: NotificationStatus.QUEUED,
    },
    providerResponse: { type: String, trim: true },
    attemptCount: { type: Number, required: true, default: 0, min: 0 },
    lastAttemptAt: { type: Date },
  },
  { timestamps: true },
)

NotificationSchema.index({ incidentId: 1, createdAt: -1 })
NotificationSchema.index({ status: 1, createdAt: -1 })

export const Notification = mongoose.model<INotification>(
  'Notifications',
  NotificationSchema,
)

```

Listing 4.19 — backend/src/models/RescueTeam.ts

Response teams with responder-only membership.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum TeamType {
  POLICE = 'Police',
  MEDICAL = 'Medical',
  FIRE = 'Fire',
  NGO = 'NGO',
  VOLUNTEER = 'Volunteer',
}

export interface IRescueTeam extends Document {
  _id: Types.ObjectId
  name: string
  teamType: TeamType
  phone: string
  email?: string
  isActive: boolean
  specializations?: string[]
  members: Types.ObjectId[]
  /**
   * Optional link to the shared Locations collection. Absent for teams
   * created before location support or for teams without a fixed base –
   * never fabricated, never defaulted.
   */
  locationId?: Types.ObjectId
  createdAt: Date
  updatedAt: Date
}

const RescueTeamSchema = new Schema<IRescueTeam>(
  {
    name: { type: String, required: true, trim: true },
    teamType: { type: String, enum: TeamType, required: true },
    phone: { type: String, required: true, trim: true },
    email: { type: String, lowercase: true, trim: true },
    isActive: { type: Boolean, required: true, default: true },
    specializations: { type: [String], default: [] },
    members: { type: [Schema.Types.ObjectId], ref: 'Users', default: [] },
    locationId: { type: Schema.Types.ObjectId, ref: 'Locations', index: true },
  },
  { timestamps: true },
)

```

```

RescueTeamSchema.index({ teamType: 1, isActive: 1 })
RescueTeamSchema.index({ name: 1 })

```

```

export const RescueTeam = mongoose.model<IRescueTeam>('RescueTeams', RescueTeamSchema)

```

Listing 4.20 — backend/src/models/RescueAssignment.ts

Incident-to-team assignments with validated workflow.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum AssignmentStatus {
  ASSIGNED = 'ASSIGNED',
  EN_ROUTE = 'EN_ROUTE',
  ON_SCENE = 'ON_SCENE',
  COMPLETED = 'COMPLETED',
  CANCELLED = 'CANCELLED',
}

export interface IRescueAssignment extends Document {
  _id: Types.ObjectId
  incidentId: Types.ObjectId
  teamId: Types.ObjectId
  assignedBy: Types.ObjectId
  assignedAt: Date
  status: AssignmentStatus
  notes?: string
  createdAt: Date
  updatedAt: Date
}

const RescueAssignmentSchema = new Schema<IRescueAssignment>(
  {
    incidentId: { type: Schema.Types.ObjectId, ref: 'Incidents', required: true, index: true },
    teamId: { type: Schema.Types.ObjectId, ref: 'RescueTeams', required: true, index: true },
    assignedBy: { type: Schema.Types.ObjectId, ref: 'Users', required: true },
    assignedAt: { type: Date, required: true, default: Date.now },
    status: {
      type: String,
      enum: AssignmentStatus,
      required: true,
      default: AssignmentStatus.ASSIGNED,
    },
    notes: { type: String, trim: true },
  },
  { timestamps: true },
)

RescueAssignmentSchema.index({ incidentId: 1, status: 1 })
RescueAssignmentSchema.index({ teamId: 1, status: 1 })
RescueAssignmentSchema.index({ assignedBy: 1 })

export const RescueAssignment = mongoose.model<IRescueAssignment>(
  'RescueAssignments',
  RescueAssignmentSchema,
)

```

Listing 4.21 — backend/src/models/Facility.ts

Emergency facility register with location links.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum FacilityType {
  HOSPITAL = 'Hospital',
  SHELTER = 'Shelter',
  POLICE_STATION = 'Police Station',
  FIRE_STATION = 'Fire Station',
  RELIEF_CENTRE = 'Relief Centre',
}

export interface IFacility extends Document {
  _id: Types.ObjectId

```

```

name: string
facilityType: FacilityType
locationId: Types.ObjectId
phone: string
capacity?: number
isOperational: boolean
operatingHours?: string
createdAt: Date
updatedAt: Date
}

const FacilitySchema = new Schema<IFacility>(
  {
    name: { type: String, required: true, trim: true },
    facilityType: { type: String, enum: FacilityType, required: true },
    locationId: { type: Schema.Types.ObjectId, ref: 'Locations', required: true, index: true },
    phone: { type: String, required: true, trim: true },
    capacity: { type: Number, min: 0 },
    isOperational: { type: Boolean, required: true, default: true },
    operatingHours: { type: String, trim: true },
  },
  { timestamps: true },
)

FacilitySchema.index({ facilityType: 1, isOperational: 1 })
FacilitySchema.index({ name: 1 })

export const Facility = mongoose.model<IFacility>('Facilities', FacilitySchema)

```

Listing 4.22 — backend/src/models/UnsafeAreaReport.ts

Community unsafe-area reports with verification flag.

```

import mongoose, { Schema, type Document, type Types } from 'mongoose'

export interface IUnsafeAreaReport extends Document {
  _id: Types.ObjectId
  reportedBy: Types.ObjectId
  locationId: Types.ObjectId
  category: string
  description: string
  severity: string
  isVerified: boolean
  createdAt: Date
  updatedAt: Date
}

const UnsafeAreaReportSchema = new Schema<IUnsafeAreaReport>(
  {
    reportedBy: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
    locationId: { type: Schema.Types.ObjectId, ref: 'Locations', required: true, index: true },
    category: { type: String, required: true, trim: true },
    description: { type: String, required: true, trim: true },
    severity: { type: String, required: true, trim: true },
    isVerified: { type: Boolean, required: true, default: false },
  },
  { timestamps: true },
)

UnsafeAreaReportSchema.index({ reportedBy: 1, createdAt: -1 })
UnsafeAreaReportSchema.index({ isVerified: 1, createdAt: -1 })

export const UnsafeAreaReport = mongoose.model<IUnsafeAreaReport>(
  'UnsafeAreaReports',
  UnsafeAreaReportSchema,
)

```

4.4 Backend — API routes

Listing 4.23 — backend/src/routes/auth.ts

Public registration, login, logout and profile routes.

```

import { Router } from 'express'
import { login, logout, me, register, updateProfile } from '../controllers/authController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'

const router = Router()

// Public auth endpoints are heavily targeted by credential stuffing;
// generous per-IP windows stop brute force without blocking legitimate use.
const registerLimiter = rateLimit({ windowMs: 60 * 60 * 1000, max: 20, message: 'Too many signups from this IP. Try again later.' })

router.post('/register', requireDb, registerLimiter, register)
router.post('/login', requireDb, rateLimit({ windowMs: 15 * 60 * 1000, max: 40, message: 'Too many login attempts. Try again in a few minutes.' })), login)
router.post('/logout', logout)
router.get('/me', requireAuth, requireDb, me)
router.patch('/profile', requireAuth, requireDb, updateProfile)

export default router

```

Listing 4.24 — backend/src/routes/incidents.ts

Owner-scoped incident, update, risk and nearby routes.

```

import { Router } from 'express'
import {
  createIncident,
  deleteIncident,
  getIncident,
  listIncidentUpdates,
  listIncidents,
  updateIncident,
} from '../controllers/incidentController.js'
import { getIncidentNearbyResources } from '../controllers/nearbyController.js'
import { assessRisk, listRisk } from '../controllers/riskController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'

const router = Router()

// ALL incident routes are user-owned: identity comes from the JWT, never the body.
// Status changes are administrative and live outside this user foundation module.
router.use(requireAuth, requireDb)

// A flood of incident reports is a SOS-poisoning vector: cap creation per IP.
router.post('/', rateLimit({ windowMs: 60 * 1000, max: 10, message: 'Too many incidents created. Please wait a moment.' })), createIncident)
router.get('/', listIncidents)
router.get('/:id', getIncident)
router.patch('/:id', updateIncident)
router.delete('/:id', deleteIncident)
router.get('/:id/updates', listIncidentUpdates)
router.post('/:id/risk', assessRisk)
router.get('/:id/risk', listRisk)
router.get('/:id/nearby-resources', getIncidentNearbyResources)

export default router

```

Listing 4.25 — backend/src/routes/admin.ts

Protected administration console routes (requireAdmin).

```

import { Router } from 'express'
import { listNotificationsAdmin, listUsers, getUserAdmin, updateUserAdmin, deleteUserAdmin } from '../controllers/adminController.js'
import {
  deleteEmergencyContactAdmin,
  listAllEmergencyContactsAdmin,
  listEmergencyContactsAdmin,
  updateEmergencyContactAdmin,
}

```

```

} from '../controllers/adminEmergencyContactController.js'
import {
  createFacility,
  getFacilityAdmin,
  listFacilitiesAdmin,
  updateFacility,
} from '../controllers/adminFacilityController.js'
import {
  getIncidentAdmin,
  getIncidentsSummary,
  listIncidentsAdmin,
  updateIncidentStatus,
} from '../controllers/adminIncidentController.js'
import {
  addTeamMember,
  createTeam,
  getTeamAdmin,
  getTeamMembers,
  listTeamsAdmin,
  removeTeamMember,
  updateTeam,
} from '../controllers/adminRescueTeamController.js'
import {
  createAssignment,
  listIncidentAssignments,
  updateAssignment,
} from '../controllers/adminAssignmentController.js'
import {
  getUnsafeReportAdmin,
  listUnsafeReportsAdmin,
  verifyUnsafeReport,
  deleteUnsafeReportAdmin,
} from '../controllers/adminUnsafeReportController.js'
import { getAdminIncidentNearbyResources } from '../controllers/nearbyController.js'
import { listRiskAdmin } from '../controllers/adminRiskController.js'
import { getDashboard } from '../controllers/adminDashboardController.js'
import {
  deleteReport,
  exportReport,
  generateReport,
  getReport,
  listReports,
} from '../controllers/adminReportController.js'
import { requireAuth, requireAdmin } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'

const router = Router()

router.use(requireAuth, requireAdmin, requireDb)

router.get('/dashboard', getDashboard)
router.post('/reports', generateReport)
router.get('/reports', listReports)
router.get('/reports/:id', getReport)
router.get('/reports/:id/export', exportReport)
router.delete('/reports/:id', deleteReport)
router.get('/users', listUsers)
router.get('/users/:id', getUserAdmin)
router.patch('/users/:id', updateUserAdmin)
router.delete('/users/:id', deleteUserAdmin)
router.get('/users/:userId/emergency-contacts', listEmergencyContactsAdmin)
router.get('/emergency-contacts', listAllEmergencyContactsAdmin)
router.patch('/emergency-contacts/:contactId', updateEmergencyContactAdmin)
router.delete('/emergency-contacts/:contactId', deleteEmergencyContactAdmin)
router.get('/notifications', listNotificationsAdmin)
router.get('/incidents/summary', getIncidentsSummary)
router.get('/incidents', listIncidentsAdmin)
router.get('/incidents/:id', getIncidentAdmin)
router.patch('/incidents/:id/status', updateIncidentStatus)
router.post('/facilities', createFacility)
router.get('/facilities', listFacilitiesAdmin)
router.get('/facilities/:id', getFacilityAdmin)
router.patch('/facilities/:id', updateFacility)
router.post('/rescue-teams', createTeam)
router.get('/rescue-teams', listTeamsAdmin)
router.get('/rescue-teams/:id', getTeamAdmin)

```

```

router.patch('/rescue-teams/:id', updateTeam)
router.get('/rescue-teams/:id/members', getTeamMembers)
router.post('/rescue-teams/:id/members', addTeamMember)
router.delete('/rescue-teams/:id/members/:userId', removeTeamMember)
router.post('/incidents/:id/assignments', createAssignment)
router.get('/incidents/:id/assignments', listIncidentAssignments)
router.patch('/assignments/:id', updateAssignment)
router.get('/unsafe-reports', listUnsafeReportsAdmin)
router.get('/unsafe-reports/:id', getUnsafeReportAdmin)
router.patch('/unsafe-reports/:id', verifyUnsafeReport)
router.delete('/unsafe-reports/:id', deleteUnsafeReportAdmin)
router.get('/incidents/:id/risk', listRiskAdmin)
router.get('/incidents/:id/nearby-resources', getAdminIncidentNearbyResources)

export default router

```

Listing 4.26 — backend/src/routes/responder.ts

Responder-scoped assignment routes (requireResponder).

```

import { Router } from 'express'
import {
  getMyIncident,
  listMyAssignments,
  listMyNotifications,
  updateMyAssignment,
} from '../controllers/responderController.js'
import { requireAuth, requireResponder } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'

const router = Router()

// Responder-owned scope: identity comes from the JWT, and every Lookup is
// constrained to teams the caller belongs to. Status changes use the same
// documented assignment workflow as the admin flow.
router.use(requireAuth, requireResponder, requireDb)

router.get('/assignments', listMyAssignments)
router.patch('/assignments/:id', updateMyAssignment)
router.get('/incidents/:id', getMyIncident)
router.get('/notifications', listMyNotifications)

export default router

```

Listing 4.27 — backend/src/routes/emergencyContacts.ts

Owner-scoped emergency-contact routes.

```

import { Router } from 'express'
import {
  createContact,
  deleteContact,
  listContacts,
  updateContact,
} from '../controllers/emergencyContactController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'

const router = Router()

// ALL contact routes are user-owned: identity comes from the JWT, never the body.
router.use(requireAuth, requireDb)

router.post('/', createContact)
router.get('/', listContacts)
router.patch('/:id', updateContact)
router.delete('/:id', deleteContact)

export default router

```

Listing 4.28 — backend/src/routes/unsafeReports.ts

Owner-scoped unsafe-area report routes.

```
import { Router } from 'express'
import { createUnsafeReport, deleteUnsafeReport, getUnsafeReport, listUnsafeReports, updateUnsafeReport }
from '../controllers/unsafeReportController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'

const router = Router()

// User-owned reports: identity comes from the JWT, never the body.
router.use(requireAuth, requireDb)

// Cap report submission per IP to keep the moderation queue honest.
router.post('/', rateLimit({ windowMs: 60 * 1000, max: 10, message: 'Too many reports submitted. Please wait
a moment.' })), createUnsafeReport)
router.get('/', listUnsafeReports)
router.get('/:id', getUnsafeReport)
router.patch('/:id', updateUnsafeReport)
router.delete('/:id', deleteUnsafeReport)

export default router
```

4.5 Backend — controllers and services

Listing 4.29 — backend/src/controllers/authController.ts

Registration, login, profile and safe user shaping.

```
import type { NextFunction, Request, Response } from 'express'
import { User, UserRole, type IUser } from '../models/User.js'
import { badRequest, conflict, notFoundError, unauthorized } from '../utils/errors.js'
import { hashPassword, comparePassword } from '../utils/password.js'
import { signAuthToken } from '../utils/jwt.js'
import {
  validateLogin,
  validateProfileUpdate,
  validateRegister,
  isEmail,
} from '../validators/auth.js'

interface SafeUser {
  id: string
  name: string
  email: string
  phone: string
  role: UserRole
  language?: string
  createdAt: Date
  updatedAt: Date
}

/** Strip internal fields (passwordHash, __v). Never expose secrets. */
export function toSafeUser(user: IUser): SafeUser {
  return {
    id: String(user._id),
    name: user.name,
    email: user.email,
    phone: user.phone,
    role: user.role,
    ...(user.language ? { language: user.language } : {}),
    createdAt: user.createdAt,
    updatedAt: user.updatedAt,
  }
}

function issueToken(user: IUser): string {
  return signAuthToken(String(user._id), user.role)
}
```

```

/**
 * POST /api/auth/register
 * Public self-registration. Always creates a USER account -
 * the role field is never accepted from the client (admins are pre-created).
 */
export async function register(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const { input, issues } = validateRegister(req.body)
    if (!input || issues) {
      next(badRequest('Invalid registration data.', issues))
      return
    }

    const emailTaken = await User.findOne({ email: input.email }).lean()
    if (emailTaken) {
      next(conflict('An account with this email already exists.'))
      return
    }
    const phoneTaken = await User.findOne({ phone: input.phone }).lean()
    if (phoneTaken) {
      next(conflict('An account with this phone number already exists.'))
      return
    }

    const passwordHash = await hashPassword(input.password)
    const user = await User.create({
      name: input.name,
      email: input.email,
      phone: input.phone,
      passwordHash,
      role: UserRole.USER,
      ...(input.language ? { language: input.language } : {}),
    })

    res.status(201).json({
      success: true,
      data: { user: toSafeUser(user), token: issueToken(user) },
    })
  } catch (err) {
    next(err)
  }
}

/**
 * POST /api/auth/login
 * Accepts an email address or phone number as identifier.
 * Returns a generic error for unknown accounts and wrong passwords alike.
 */
export async function login(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const { input, issues } = validateLogin(req.body)
    if (!input || issues) {
      next(badRequest('Invalid login data.', issues))
      return
    }

    const identifier = input.identifier.trim()
    let user: IUser | null
    if (isEmail(identifier)) {
      user = await User.findOne({ email: identifier.toLowerCase() })
    } else {
      // Administrators and imported accounts may store their phone with a
      // +91 prefix while the login form submits the normalized 10-digit form
      // (mirrors the frontend's normalizePhone). Match every variant.
      const normalized = identifier.replace(/^+91[\s-]?/, '').replace(/[\s-]/g, '')
      user = await User.findOne({
        $or: [
          { phone: identifier },
          { phone: normalized },
          { phone: normalized.length === 10 ? `+91${normalized}` : normalized },
        ],
      })
    }
  }
}

```

```

    if (!user) {
      next(unauthorized('Invalid credentials.'))
      return
    }
    if (user.isActive === false) {
      next(unauthorized('Invalid credentials.'))
      return
    }
    const ok = await comparePassword(input.password, user.passwordHash)
    if (!ok) {
      next(unauthorized('Invalid credentials.'))
      return
    }

    res.json({
      success: true,
      data: { user: toSafeUser(user), token: issueToken(user) },
    })
  } catch (err) {
    next(err)
  }
}

/**
 * POST /api/auth/logout
 * JWTs are stateless: Logout is completed by discarding the token client-side.
 * This endpoint acknowledges the logout so clients have a single contract.
 */
export async function logout(_req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    res.json({ success: true, data: { message: 'Logged out. Discard the stored token.' } })
  } catch (err) {
    next(err)
  }
}

/** GET /api/auth/me - current authenticated profile. */
export async function me(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const user = await User.findById(req.auth?.userId).select('-passwordHash')
    if (!user) {
      next(notFoundError('Account not found.'))
      return
    }
    res.json({ success: true, data: { user: toSafeUser(user) } })
  } catch (err) {
    next(err)
  }
}

/** PATCH /api/auth/profile - update permitted profile fields. */
export async function updateProfile(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const { input, issues } = validateProfileUpdate(req.body)
    if (!input || issues) {
      next(badRequest('Invalid profile data.', issues))
      return
    }

    const userId = req.auth?.userId
    const user = await User.findById(userId)
    if (!user) {
      next(notFoundError('Account not found.'))
      return
    }

    if (input.email && input.email !== user.email) {
      const taken = await User.findOne({ email: input.email, _id: { $ne: user._id } }).lean()
      if (taken) {
        next(conflict('An account with this email already exists.'))
        return
      }
    }
    user.email = input.email
  }
}

```

```

    if (input.phone && input.phone !== user.phone) {
      const taken = await User.findOne({ phone: input.phone, _id: { $ne: user._id } }).lean()
      if (taken) {
        next(conflict('An account with this phone number already exists.'))
        return
      }
      user.phone = input.phone
    }
    if (input.name) user.name = input.name
    if (input.language !== undefined) user.language = input.language

    await user.save()
    res.json({ success: true, data: { user: toSafeUser(user) } })
  } catch (err) {
    next(err)
  }
}

```

Listing 4.30 — backend/src/controllers/incidentController.ts

SOS creation, owner-scoped reads, edits and deletes.

```

import type { NextFunction, Request, Response } from 'express'
import { Incident, IncidentStatus, type IIncident } from '../models/Incident.js'
import { Location, type ILocation } from '../models/Location.js'
import { resolveAndStoreAddress } from '../services/geocoding.js'
import { IncidentUpdate } from '../models/IncidentUpdate.js'
import { RescueAssignment } from '../models/RescueAssignment.js'
import { attachTeams } from './adminAssignmentController.js'
import { badRequest, conflict, notFoundError, unauthorized } from '../utils/errors.js'
import { recordEventSafe } from '../services/notifications.js'
import { isValidObjectId } from '../validators/emergencyContact.js'
import { validateIncidentCreate, validateIncidentUpdate } from '../validators/incident.js'

interface SafeIncident {
  id: string
  userId: string
  type: string
  category: string
  description: string
  priority: string
  status: string
  locationId?: string
  createdAt: Date
  updatedAt: Date
  resolvedAt?: Date
}

export interface SafeLocation {
  id: string
  latitude: number
  longitude: number
  address?: string
  city?: string
  district?: string
  state?: string
  postalCode?: string
  country?: string
  accuracy?: number
  /** Capture time (the record's creation time) – shown as "Captured". */
  capturedAt: Date
}

/** Owner-scoped Location payload – only ever served alongside its own incident. */
export function toSafeLocation(doc: ILocation): SafeLocation {
  return {
    id: String(doc._id),
    latitude: doc.latitude,
    longitude: doc.longitude,
    ...(doc.address ? { address: doc.address } : {}),
    ...(doc.city ? { city: doc.city } : {}),
    ...(doc.district ? { district: doc.district } : {}),
    ...(doc.state ? { state: doc.state } : {}),
    ...(doc.postalCode ? { postalCode: doc.postalCode } : {}),
    ...(doc.country ? { country: doc.country } : {}),
  }
}

```

```

    ...(doc.accuracy !== undefined ? { accuracy: doc.accuracy } : {}),
    capturedAt: doc.createdAt,
  }
}

export function toSafeIncident(doc: IIIncident): SafeIncident {
  return {
    id: String(doc._id),
    userId: String(doc.userId),
    type: doc.type,
    category: doc.category,
    description: doc.description,
    priority: doc.priority,
    status: doc.status,
    ...(doc.locationId ? { locationId: String(doc.locationId) } : {}),
    createdAt: doc.createdAt,
    updatedAt: doc.updatedAt,
    ...(doc.resolvedAt ? { resolvedAt: doc.resolvedAt } : {}),
  }
}

function requireOwnerId(req: Request): string {
  const userId = req.auth?.userId
  if (!userId) throw unauthorized('Authentication required.')
  return userId
}

/**
 * POST /api/incidents
 * Owner and initial status are server-owned: userId comes from the JWT and
 * status is always REPORTED. A client-supplied status/userId is never trusted.
 */
export async function createIncident(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { input, issues } = validateIncidentCreate(req.body)
    if (!input || issues) {
      next(badRequest('Invalid incident data.', issues))
      return
    }

    let locationId = input.locationId
    if (input.location) {
      const created = await Location.create(input.location)
      locationId = String(created._id)
    } else if (locationId) {
      const exists = await Location.findById(locationId).select('_id').lean()
      if (!exists) {
        next(badRequest('Invalid incident data.', [{ field: 'locationId', message: 'Location not found.' }]))
        return
      }
    }

    const doc = await Incident.create({
      userId: ownerId,
      type: input.type,
      category: input.category,
      description: input.description,
      priority: input.priority,
      status: IncidentStatus.REPORTED,
      ...(locationId ? { locationId } : {}),
    })

    // Real event, recorded only because creation succeeded. Best-effort:
    // a notification write failure never fails the incident itself.
    await recordEventSafe(String(doc._id), ownerId, { kind: 'incident.created' })

    res.status(201).json({ success: true, data: { incident: toSafeIncident(doc) } })
  } catch (err) {
    next(err)
  }
}

```

```

/** GET /api/incidents – only the caller's own incidents, newest first. */
export async function listIncidents(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const docs = await Incident.find({ userId: ownerId }).sort({ createdAt: -1 }).limit(200)
    res.json({ success: true, data: { incidents: docs.map(toSafeIncident) } })
  } catch (err) {
    next(err)
  }
}

/**
 * GET /api/incidents/:id
 * Scoped lookup `{ _id, userId }`: another user's id yields 404, never their data.
 * Includes the linked location record and the visible assignment information
 * (team name/type/phone + status) for the caller's own incident.
 */
export async function getIncident(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const doc = await Incident.findOne({ _id: id, userId: ownerId })
    if (!doc) {
      next(notFoundError('Incident not found.'))
      return
    }
    const [location, assignmentDocs] = await Promise.all([
      doc.locationId ? Location.findById(doc.locationId) : Promise.resolve(null),
      RescueAssignment.find({ incidentId: doc._id }).sort({ createdAt: -1 }),
    ])
    // Backfill a readable address once per location (best-effort, async):
    // the response is not delayed, and the live reverse-geocode endpoint
    // covers display until the stored record is enriched.
    if (location && !location.address && doc.locationId) {
      void resolveAndStoreAddress(String(doc.locationId))
    }
    // User view carries only response-facing fields: no assignedBy, no internal notes.
    const assignments = (await attachTeams(assignmentDocs)).map((a) => ({
      id: a.id,
      team: a.team,
      status: a.status,
      assignedAt: a.assignedAt,
    })))
    res.json({
      success: true,
      data: {
        incident: toSafeIncident(doc),
        location: location ? toSafeLocation(location) : null,
        assignments,
      },
    })
  } catch (err) {
    next(err)
  }
}

/**
 * GET /api/incidents/:id/updates – chronological history of the caller's own
 * incident, oldest first. Scoped lookup: another user's id yields 404.
 */
export async function listIncidentUpdates(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const doc = await Incident.findOne({ _id: id, userId: ownerId }).select('_id')

```

```

if (!doc) {
  next(notFoundError('Incident not found.'))
  return
}
const updates = await IncidentUpdate.find({ incidentId: doc._id }).sort({ createdAt: 1 })
res.json({
  success: true,
  data: {
    updates: updates.map((u) => ({
      id: String(u._id),
      statusFrom: u.statusFrom ?? null,
      statusTo: u.statusTo,
      ...(u.comment ? { comment: u.comment } : {}),
      updatedBy: String(u.updatedBy),
      createdAt: u.createdAt,
    })),
  },
})
} catch (err) {
  next(err)
}
}

/**
 * True while the caller may still manage the case themselves: the incident is
 * still new (REPORTED/ACKNOWLEDGED, nothing responding yet) and no rescue team
 * has ever been tied to it. Status is independent of assignment creation, so
 * both conditions must hold. Past this window edits/deletes are declined.
 */
async function isCaseManageable(doc: IIIncident): Promise<boolean> {
  const inWindow =
    doc.status === IncidentStatus.REPORTED || doc.status === IncidentStatus.ACKNOWLEDGED
  if (!inWindow) return false
  const hasAssignments = await RescueAssignment.exists({ incidentId: doc._id })
  return !hasAssignments
}

const CASE_MANAGE_EXPLANATION =
  'Editing/deletion is only available for your new, unassigned cases (REPORTED or ACKNOWLEDGED).'

/**
 * PATCH /api/incidents/:id - owner-scoped edit. Only the four user-supplied
 * case fields (type/category/description/priority) may change; status, ownership
 * and the stored location are never taken from the request body. Declined with
 * 409 once the case is no longer new or a rescue team is involved.
 */
export async function updateIncident(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const { input, issues } = validateIncidentUpdate(req.body)
    if (!input || issues) {
      next(badRequest('Invalid incident data.', issues))
      return
    }

    const doc = await Incident.findOne({ _id: id, userId: ownerId })
    if (!doc) {
      next(notFoundError('Incident not found.'))
      return
    }
    if (!(await isCaseManageable(doc))) {
      next(conflict(CASE_MANAGE_EXPLANATION))
      return
    }

    doc.type = input.type ?? doc.type
    doc.category = input.category ?? doc.category
    doc.description = input.description ?? doc.description
  }
}

```

```

    doc.priority = input.priority ?? doc.priority
    await doc.save()

    res.json({ success: true, data: { incident: toSafeIncident(doc) } })
  } catch (err) {
    next(err)
  }
}

/**
 * DELETE /api/incidents/:id - owner-scoped delete of the case record only.
 * Declined with 409 while the case is still being worked or a rescue team was
 * ever involved. Related records (location, notifications, updates) are left
 * untouched - the backend performs no cascading deletes.
 */
export async function deleteIncident(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const doc = await Incident.findOne({ _id: id, userId: ownerId })
    if (!doc) {
      next(notFoundError('Incident not found.'))
      return
    }
    if (!(await isCaseManageable(doc))) {
      next(conflict(CASE_MANAGE_EXPLANATION))
      return
    }

    await Incident.deleteOne({ _id: doc._id })
    res.json({ success: true, data: { deleted: true, id: String(doc._id) } })
  } catch (err) {
    next(err)
  }
}

```

Listing 4.31 — backend/src/controllers/riskController.ts

Risk assessment trigger and history for owners.

```

import type { NextFunction, Request, Response } from 'express'
import { Incident } from '../models/Incident.js'
import { Location } from '../models/Location.js'
import { RiskAssessment, type IRiskAssessment } from '../models/RiskAssessment.js'
import { assessIncident } from '../services/riskAssessment.js'
import { badRequest, notFoundError, unauthorized } from '../utils/errors.js'
import { isValidObjectId } from '../validators/emergencyContact.js'

export interface SafeRiskAssessment {
  id: string
  locationId: string
  riskScore: number
  riskLevel: string
  modelVersion: string
  inputFactors: Record<string, unknown>[]
  assessedAt: Date
  createdAt: Date
}

export function toSafeAssessment(doc: IRiskAssessment): SafeRiskAssessment {
  return {
    id: String(doc._id),
    locationId: String(doc.locationId),
    riskScore: doc.riskScore,
    riskLevel: doc.riskLevel,
    modelVersion: doc.modelVersion,
    inputFactors: doc.inputFactors ?? [],
    assessedAt: doc.assessedAt,
    createdAt: doc.createdAt,
  }
}

```

```

    }
  }

function requireOwnerId(req: Request): string {
  const userId = req.auth?.userId
  if (!userId) throw unauthorized('Authentication required.')
  return userId
}

/**
 * POST /api/incidents/:id/risk - assistive assessment of the caller's own
 * incident. Requires stored coordinates (the schema ties assessments to
 * Locations). The incident itself is never modified; a failed scoring
 * attempt stores nothing and reports an error instead of success.
 */
export async function assessRisk(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const incident = await Incident.findOne({ _id: id, userId: ownerId })
    if (!incident) {
      next(notFoundError('Incident not found.'))
      return
    }
    if (!incident.locationId) {
      next(badRequest('Risk assessment needs stored coordinates.', [{ field: 'locationId', message: 'This incident has no location.' }]))
      return
    }
    const location = await Location.findById(incident.locationId)
    if (!location) {
      next(badRequest('Risk assessment needs stored coordinates.', [{ field: 'locationId', message: 'Location not found.' }]))
      return
    }

    const doc = await assessIncident(incident, location)
    res.status(201).json({ success: true, data: { assessment: toSafeAssessment(doc) } })
  } catch (err) {
    next(err)
  }
}

/**
 * GET /api/incidents/:id/risk - past assessments of the caller's own
 * incident, newest first. Ownership checked before anything is returned.
 */
export async function listRisk(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }

    const incident = await Incident.findOne({ _id: id, userId: ownerId }).select('_id locationId')
    if (!incident) {
      next(notFoundError('Incident not found.'))
      return
    }
    if (!incident.locationId) {
      res.json({ success: true, data: { assessments: [] } })
      return
    }
    const docs = await RiskAssessment.find({ locationId: incident.locationId }).sort({ assessedAt: -1 })
    res.json({ success: true, data: { assessments: docs.map(toSafeAssessment) } })
  } catch (err) {
    next(err)
  }
}

```

```
}
}
```

Listing 4.32 — backend/src/controllers/adminRiskController.ts

Same stored assessment served to administrators.

```
import type { NextFunction, Request, Response } from 'express'
import { Incident } from '../models/Incident.js'
import { RiskAssessment } from '../models/RiskAssessment.js'
import { badRequest, notFoundError } from '../utils/errors.js'
import { isValidObjectId } from '../validators/emergencyContact.js'
import { toSafeAssessment } from './riskController.js'

/** GET /api/admin/incidents/:id/risk - assessments for one incident, newest first. */
export async function listRiskAdmin(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const { id } = req.params
    if (!isValidObjectId(id)) {
      next(badRequest('Invalid incident id.'))
      return
    }
    const incident = await Incident.findById(id).select('_id locationId')
    if (!incident) {
      next(notFoundError('Incident not found.'))
      return
    }
    if (!incident.locationId) {
      res.json({ success: true, data: { assessments: [] } })
      return
    }
    const docs = await RiskAssessment.find({ locationId: incident.locationId }).sort({ assessedAt: -1 })
    res.json({ success: true, data: { assessments: docs.map(toSafeAssessment) } })
  } catch (err) {
    next(err)
  }
}
```

Listing 4.33 — backend/src/services/riskAssessment.ts

Factor assembly, AI call, validation and storage.

```
import { env } from '../config/env.js'
import { Incident, IncidentStatus, type IIIncident } from '../models/Incident.js'
import { Location, type ILocation } from '../models/Location.js'
import { RiskAssessment, RiskLevel } from '../models/RiskAssessment.js'
import { UnsafeAreaReport } from '../models/UnsafeAreaReport.js'
import { HttpError } from '../utils/errors.js'

/** Nearby radius in kilometres for contextual risk factors. */
export const NEARBY_KM = 2

const ACTIVE_INCIDENT_STATUSES = [
  IncidentStatus.REPORTED,
  IncidentStatus.ACKNOWLEDGED,
  IncidentStatus.ASSIGNED,
  IncidentStatus.IN_PROGRESS,
]

export interface RiskFactors {
  priority: string
  incidentType: string
  verifiedReports: number
  unverifiedReports: number
  activeIncidents: number
}

export interface AiRiskResult {
  riskScore: number
  riskLevel: RiskLevel
  modelVersion: string
  inputFactors: Record<string, unknown>[]
  assessedAt: Date
}
```

```

function haversineKm(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const toRad = (d: number): number => (d * Math.PI) / 180
  const earthKm = 6371
  const a =
    Math.sin(toRad(lat2 - lat1) / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(toRad(lon2 - lon1) / 2) ** 2
  return 2 * earthKm * Math.asin(Math.sqrt(a))
}

/**
 * Assemble scoring factors purely from stored records around the incident's
 * location. Free-text descriptions are never sent anywhere – only bounded,
 * enumerated values leave this function.
 */
export async function assembleFactors(incident: IIncident, location: ILocation): Promise<RiskFactors> {
  const [reports, activeOthers] = await Promise.all([
    UnsafeAreaReport.find().select('locationId isVerified'),
    Incident.find({
      _id: { $ne: incident._id },
      status: { $in: ACTIVE_INCIDENT_STATUSES },
    }).select('locationId'),
  ])

  const locationIds = new Set<string>()
  for (const r of reports) locationIds.add(String(r.locationId))
  for (const i of activeOthers) {
    if (i.locationId) locationIds.add(String(i.locationId))
  }
  const locs = locationIds.size > 0 ? await Location.find({ _id: { $in: [...locationIds] } }) : []
  const coords = new Map(locs.map((l) => [String(l._id), l]))

  const within = (locationId: unknown): boolean => {
    const l = coords.get(String(locationId))
    if (!l) return false
    return haversineKm(location.latitude, location.longitude, l.latitude, l.longitude) <= NEARBY_KM
  }

  let verifiedReports = 0
  let unverifiedReports = 0
  for (const r of reports) {
    if (!within(r.locationId)) continue
    if (r.isVerified) verifiedReports += 1
    else unverifiedReports += 1
  }
  let activeIncidents = 0
  for (const i of activeOthers) {
    if (i.locationId && within(i.locationId)) activeIncidents += 1
  }

  return {
    priority: incident.priority,
    incidentType: incident.type,
    verifiedReports,
    unverifiedReports,
    activeIncidents,
  }
}

function validateAiResult(body: unknown): AiRiskResult {
  const o = (typeof body === 'object' && body !== null ? body : {}) as Record<string, unknown>
  const levels = Object.values(RiskLevel) as string[]

  if (
    typeof o.riskScore !== 'number' ||
    !Number.isInteger(o.riskScore) ||
    o.riskScore < 0 ||
    o.riskScore > 100
  ) {
    throw new HttpError(502, 'AI service returned an invalid risk score.')
  }
  if (typeof o.riskLevel !== 'string' || !levels.includes(o.riskLevel)) {
    throw new HttpError(502, 'AI service returned an invalid risk level.')
  }
}

```

```

}
if (typeof o.modelVersion !== 'string' || o.modelVersion.trim() === '' || o.modelVersion.length > 50) {
  throw new HttpError(502, 'AI service returned an invalid model version.')
}
if (!Array.isArray(o.inputFactors)) {
  throw new HttpError(502, 'AI service returned invalid input factors.')
}
const assessedAt = new Date(typeof o.assessedAt === 'string' ? o.assessedAt : NaN)
if (Number.isNaN(assessedAt.getTime())) {
  throw new HttpError(502, 'AI service returned an invalid assessment time.')
}

return {
  riskScore: o.riskScore,
  riskLevel: o.riskLevel as RiskLevel,
  modelVersion: o.modelVersion.trim(),
  inputFactors: o.inputFactors as Record<string, unknown>[],
  assessedAt,
}
}

/**
 * Call the configured AI service and strictly validate its response.
 * Throws 502 (bad/unreachable service) or 504 (timeout). Never stores.
 * No credentials are involved – the service needs no API key.
 */
export async function callAiService(factors: RiskFactors): Promise<AiRiskResult> {
  const url = `${env.aiServiceUrl.replace(/\/+$/, '')}/risk/assess`
  let res: Response
  try {
    res = await fetch(url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(factors),
      signal: AbortSignal.timeout(env.aiServiceTimeoutMs),
    })
  } catch (err) {
    if (err instanceof DOMException && err.name === 'TimeoutError') {
      throw new HttpError(504, 'AI service timed out. Please try again later.')
    }
    throw new HttpError(502, 'AI service is unavailable. Please try again later.')
  }

  if (!res.ok) {
    throw new HttpError(502, 'AI service could not score these factors.')
  }
  let body: unknown
  try {
    body = (await res.json()) as unknown
  } catch {
    throw new HttpError(502, 'AI service returned an unreadable response.')
  }
  return validateAiResult(body)
}

/**
 * Full assessment flow for one incident: assemble real factors, score via the
 * AI service, validate, then store. The incident itself is never modified.
 */
export async function assessIncident(incident: IIIncident, location: ILocation) {
  const factors = await assembleFactors(incident, location)
  const result = await callAiService(factors)
  const doc = await RiskAssessment.create({
    locationId: location._id,
    riskScore: result.riskScore,
    riskLevel: result.riskLevel,
    modelVersion: result.modelVersion,
    inputFactors: result.inputFactors,
    assessedAt: result.assessedAt,
  })
  return doc
}

```

4.6 AI service (Python + FastAPI)

Listing 4.34 — ai-service/app/main.py

FastAPI application with health and risk endpoints.

```
"""RakshaSafe AI service — Python + FastAPI.

Provides:
- health endpoint
- CORS + root information
- POST /risk/assess — deterministic assistive risk scoring
  (see app.risk for the documented model; decision support only).
"""

from datetime import datetime

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config import settings
from app.risk import RiskFactors, RiskResult, assess

app = FastAPI(
    title="RakshaSafe AI Service",
    description="AI-related processing for RakshaSafe (Python + FastAPI).",
    version="0.1.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/", tags=["meta"])
def root() -> dict:
    return {
        "service": "RakshaSafe AI Service",
        "version": "0.1.0",
        "docs": "/docs",
    }

@app.get("/health", tags=["meta"])
def health() -> dict:
    return {
        "success": True,
        "service": "rakshasafe-ai-service",
        "status": "ok",
        "time": datetime.utcnow().isoformat() + "Z",
    }

@app.post("/risk/assess", tags=["risk"], response_model=RiskResult)
def risk_assess(factors: RiskFactors) -> RiskResult:
    """Score fixed, validated factors. Invalid input is rejected (422) and
    never stored — storage is the Node.js backend's responsibility."""
    return assess(factors)
```

Listing 4.35 — ai-service/app/risk.py

Deterministic raksha-risk-v1 scoring engine and schemas.

```
"""Deterministic risk-scoring engine for RakshaSafe.

This is the documented assistive analysis component (Python + FastAPI AI
```

service – no external LLM or API key is involved or required):

- Input is a fixed set of numeric/categorical factors assembled by the Node.js backend from real database records. Free-text descriptions are NEVER interpreted here, so prompt-injection style input cannot alter the outcome (text arrives only as pre-computed, bounded numbers).
- Output is a strict shape: riskScore (0-100), riskLevel (LOW / MEDIUM / HIGH / CRITICAL), modelVersion, inputFactors, assessedAt.
- The result is decision support only: it predicts nothing, dispatches nothing, and must always be presented with a disclaimer.

Model ``raksha-risk-v1`` weights (documented here, versioned by MODEL_VERSION):

- priority base: LOW=15, MEDIUM=35, HIGH=60, CRITICAL=80
- disaster type modifier: +5 for Disaster, +0 for Safety
- verified unsafe reports nearby: +6 each, capped at +30
- unverified unsafe reports nearby: +2 each, capped at +10
- other active incidents nearby: +3 each, capped at +15
- total clamped to 0-100
- level bands: <25 LOW, <50 MEDIUM, <75 HIGH, otherwise CRITICAL

```
from datetime import datetime, timezone
```

```
from pydantic import BaseModel, Field
```

```
MODEL_VERSION = "raksha-risk-v1"
```

```
PRIORITY_BASE: dict[str, int] = {  
    "LOW": 15,  
    "MEDIUM": 35,  
    "HIGH": 60,  
    "CRITICAL": 80,  
}
```

```
TYPE_MODIFIER: dict[str, int] = {  
    "Safety": 0,  
    "Disaster": 5,  
}
```

```
VERIFIED_REPORT_WEIGHT = 6  
VERIFIED_REPORT_CAP = 30  
UNVERIFIED_REPORT_WEIGHT = 2  
UNVERIFIED_REPORT_CAP = 10  
ACTIVE_INCIDENT_WEIGHT = 3  
ACTIVE_INCIDENT_CAP = 15
```

```
class RiskFactors(BaseModel):  
    """Fixed factor schema. Unknown fields are ignored by pydantic default."""
```

```
    priority: str = Field(pattern="^(LOW|MEDIUM|HIGH|CRITICAL)$")  
    incidentType: str = Field(pattern="^(Safety|Disaster)$")  
    verifiedReports: int = Field(ge=0, le=1000)  
    unverifiedReports: int = Field(ge=0, le=1000)  
    activeIncidents: int = Field(ge=0, le=1000)
```

```
class RiskResult(BaseModel):  
    riskScore: int = Field(ge=0, le=100)  
    riskLevel: str = Field(pattern="^(LOW|MEDIUM|HIGH|CRITICAL)$")  
    modelVersion: str = Field(min_length=1, max_length=50)  
    inputFactors: list[dict] = Field(default_factory=list)  
    assessedAt: str = Field(min_length=1, max_length=50)
```

```
def level_for_score(score: int) -> str:  
    if score < 25:  
        return "LOW"  
    if score < 50:  
        return "MEDIUM"
```

```

if score < 75:
    return "HIGH"
return "CRITICAL"

def assess(factors: RiskFactors) -> RiskResult:
    """Pure deterministic scoring. No I/O, no randomness, no text output."""
    base = PRIORITY_BASE[factors.priority]
    type_mod = TYPE_MODIFIER[factors.incidentType]
    verified = min(factors.verifiedReports * VERIFIED_REPORT_WEIGHT, VERIFIED_REPORT_CAP)
    unverified = min(factors.unverifiedReports * UNVERIFIED_REPORT_WEIGHT, UNVERIFIED_REPORT_CAP)
    active = min(factors.activeIncidents * ACTIVE_INCIDENT_WEIGHT, ACTIVE_INCIDENT_CAP)

    score = max(0, min(100, base + type_mod + verified + unverified + active))

    input_factors = [
        {"factor": "priorityBase", "value": factors.priority, "weight": base},
        {"factor": "typeModifier", "value": factors.incidentType, "weight": type_mod},
        {"factor": "verifiedReports", "value": factors.verifiedReports, "weight": verified},
        {"factor": "unverifiedReports", "value": factors.unverifiedReports, "weight": unverified},
        {"factor": "activeIncidents", "value": factors.activeIncidents, "weight": active},
    ]

    return RiskResult(
        riskScore=score,
        riskLevel=level_for_score(score),
        modelVersion=MODEL_VERSION,
        inputFactors=input_factors,
        assessedAt=datetime.now(timezone.utc).isoformat(),
    )

```

Listing 4.36 — ai-service/app/config.py

Service configuration (CORS, ports, environment).

```

"""Configuration for the RakshaSafe AI service.

Settings are read from environment variables or a local .env file.
"""

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    service_name: str = "rakshasafe-ai-service"
    version: str = "0.1.0"
    host: str = "0.0.0.0"
    port: int = 8000
    log_level: str = "info"

    cors_origins: list[str] = ["http://localhost:5173"]

settings = Settings()

```

4.7 Frontend (React + TypeScript + Vite)

Listing 4.37 — frontend/src/App.tsx

Application shell: hash routing, guards and role redirects.

```

import { useEffect, useState } from 'react'
import { ToastProvider } from './components/ui/Toast'
import { RequireAdmin, RequireAuth, RequireResponder } from './components/auth/guards'
import { Header } from './components/layout/Header'
import { Navbar, type NavItem } from './components/layout/Navbar'
import { Badge } from './components/ui/Badge'
import { Button } from './components/ui/Button'

```

```

import { Logo } from './components/ui/Logo'
import { Spinner } from './components/ui/Spinner'
import { ActivityIcon, AlertTriangleIcon, BellIcon, BuildingIcon, FileTextIcon, HomeIcon, ListIcon,
MapPinIcon, PhoneIcon, ShieldIcon, UserIcon, UsersIcon } from './components/ui/icons'
import { api } from './lib/api'
import { useAuth } from './lib/auth-context'
import { AuthProvider } from './lib/AuthProvider'
import { ThemeProvider } from './lib/theme'
import { navigateTo, parseHash, useHashRoute, type AppRoute } from './lib/hash-route'
import { I18nProvider, useI18n } from './lib/i18n'
import { getLastSeen, isUnread, type NotificationItem } from './lib/notifications'
import { AdminFacilitiesPage } from './pages/AdminFacilitiesPage'
import { AdminEmergencyContactsPage } from './pages/AdminEmergencyContactsPage'
import { AdminIncidentDetailPage } from './pages/AdminIncidentDetailPage'
import { AdminIncidentsPage } from './pages/AdminIncidentsPage'
import { AdminTeamsPage } from './pages/AdminTeamsPage'
import { AdminUsersPage } from './pages/AdminUsersPage'
import { AdminUserDetailPage } from './pages/AdminUserDetailPage'
import { DashboardPage } from './pages/DashboardPage'
import { AdminDashboardPage } from './pages/AdminDashboardPage'
import { AdminUnsafeReportsPage } from './pages/AdminUnsafeReportsPage'
import { IncidentDetailPage } from './pages/IncidentDetailPage'
import { NotificationsPage } from './pages/NotificationsPage'
import { ReportsPage } from './pages/ReportsPage'
import { ResourcesPage } from './pages/ResourcesPage'
import { UnsafeReportsPage } from './pages/UnsafeReportsPage'
import { EmergencyContactsPage } from './pages/EmergencyContactsPage'
import { LoginPage } from './pages/LoginPage'
import { ResponderDashboardPage } from './pages/ResponderDashboardPage'
import { SosPage } from './pages/SosPage'
import { ProfilePage } from './pages/ProfilePage'
import { RegisterPage } from './pages/RegisterPage'

function Shell() {
  const route = useHashRoute()
  const { user, initializing, busy, logout } = useAuth()
  const { t } = useI18n()
  const [unreadCount, setUnreadCount] = useState(0)

  const getRoleLabel = (role: 'USER' | 'ADMIN' | 'RESPONDER') => {
    switch (role) {
      case 'ADMIN':
        return t('role.admin')
      case 'RESPONDER':
        return t('role.responder')
      default:
        return t('role.user')
    }
  }
  const roleLabel = user ? getRoleLabel(user.role) : ''

  useEffect(() => {
    // The bell unmounts on logout, so no synchronous reset is needed here;
    // the count refreshes from the backend on every sign-in and navigation.
    if (!user || initializing) {
      return
    }
    let cancelled = false
    async function fetchUnread(): Promise<void> {
      try {
        const res = await api<{ notifications: NotificationItem[] }>('/notifications')
        if (cancelled || !user) return
        const cursor = getLastSeen(user.id)
        setUnreadCount(res.notifications.filter((n) => isUnread(n, cursor)).length)
      } catch {
        if (!cancelled) setUnreadCount(0)
      }
    }
    void fetchUnread()
    return () => {
      cancelled = true
    }
  }, [user, initializing, route])

  useEffect(() => {

```

```

    if (initializing) return
    // Decide from the live hash, not the possibly-stale route state: right
    // after an auth transition a queued navigation may not have rendered yet,
    // and redirecting on stale state would clobber it.
    const live = parseHash()
    if (!user && live !== '/login' && live !== '/register') {
      navigateTo('/login')
    } else if (user && (live === '/login' || live === '/register')) {
      navigateTo(user.role === 'ADMIN' ? '/admin/dashboard' : user.role === 'RESPONDER' ? '/responder' :
'/dashboard')
    } else if (user && user.role === 'RESPONDER' && (live === '/dashboard' || live === '/sos' ||
live.startsWith('/admin/'))) {
      navigateTo('/responder')
    } else if (user && live === '/responder' && user.role === 'USER') {
      navigateTo('/dashboard')
    } else if (user && live === '/responder' && user.role === 'ADMIN') {
      navigateTo('/admin/dashboard')
    } else if (user && user.role !== 'ADMIN' && live.startsWith('/admin/')) {
      navigateTo('/dashboard')
    }
  }, [initializing, user, route])

  async function handleLogout(): Promise<void> {
    await logout()
    navigateTo('/login')
  }

  const items: NavItem[] = []
  if (user && user.role === 'RESPONDER') {
    items.push({ label: t('nav.assignments'), href: '#/responder', icon: <ShieldIcon />, active: route ===
'/responder' })
    items.push({ label: t('nav.myProfile'), href: '#/profile', icon: <HomeIcon />, active: route ===
'/profile' })
  } else if (user) {
    items.push({ label: t('nav.dashboard'), href: '#/dashboard', icon: <HomeIcon />, active: route ===
'/dashboard' })
    items.push({ label: t('nav.sos'), href: '#/sos', icon: <ShieldIcon />, active: route === '/sos' })
    items.push({ label: t('nav.contacts'), href: '#/contacts', icon: <PhoneIcon />, active: route ===
'/contacts' })
    items.push({ label: t('nav.resources'), href: '#/resources', icon: <MapPinIcon />, active: route ===
'/resources' })
    items.push({ label: t('nav.reportUnsafe'), href: '#/report-unsafe', icon: <AlertTriangleIcon />, active:
route === '/report-unsafe' })
    items.push({ label: t('nav.notifications'), href: '#/notifications', icon: <BellIcon />, active: route
=== '/notifications' })
    items.push({ label: t('nav.myProfile'), href: '#/profile', icon: <HomeIcon />, active: route ===
'/profile' })
    if (user.role === 'ADMIN') {
      items.push({ label: t('nav.overview'), href: '#/admin/dashboard', icon: <HomeIcon />, active: route ===
'/admin/dashboard' })
      items.push({
        label: t('nav.incidents'),
        href: '#/admin/incidents',
        icon: <ActivityIcon />,
        active: route === '/admin/incidents' || route === '/admin/incident-detail',
      })
      items.push({ label: t('nav.facilities'), href: '#/admin/facilities', icon: <BuildingIcon />, active:
route === '/admin/facilities' })
      items.push({ label: t('nav.teams'), href: '#/admin/teams', icon: <UserIcon />, active: route ===
'/admin/teams' })
      items.push({
        label: t('nav.adminEmergencyContacts'),
        href: '#/admin/emergency-contacts',
        icon: <PhoneIcon />,
        active: route === '/admin/emergency-contacts',
      })
      items.push({ label: t('nav.unsafeAreas'), href: '#/admin/unsafe-reports', icon: <ListIcon />, active:
route === '/admin/unsafe-reports' })
      items.push({ label: t('nav.reports'), href: '#/admin/reports', icon: <FileTextIcon />, active: route
=== '/admin/reports' })
      items.push({ label: t('nav.users'), href: '#/admin/users', icon: <UsersIcon />, active: route ===
'/admin/users' })
    }
  }
}

return (

```

```

<div className="min-h-svh">
  <Header
    nav={<Navbar items={items} />}
    showNotifications={false}
    actions={
      initializing ? (
        <Spinner size="sm" />
      ) : user ? (
        <div className="flex items-center gap-2">
          <a
            href="#/notifications"
            aria-label={unreadCount > 0 ? t('notification.unreadCount', { count: unreadCount }) :
t('nav.notifications')}
            className="relative inline-flex size-10 items-center justify-center rounded-xl text-ink-700
transition-colors hover:bg-ink-100"
          >
            <BellIcon className="size-5" />
            {unreadCount > 0 && (
              <span className="absolute right-1.5 top-1.5 flex size-4 items-center justify-center
rounded-full bg-rose-500 text-[10px] font-bold text-white dark:text-ink-950">
                {unreadCount > 9 ? '9+' : unreadCount}
              </span>
            )}
          </a>
          <span className="hidden text-sm font-semibold text-ink-700 sm:inline">{user.name}</span>
          <span className="hidden sm:inline">
            <Badge variant={user.role === 'ADMIN' ? 'secondary' : 'primary'}>{roleLabel}</Badge>
          </span>
          <Button size="sm" variant="outline" onClick={() => void handleLogout()} disabled={busy}>
            {t('nav.logout')}
          </Button>
        </div>
      ) : (
        <div className="flex items-center gap-2">
          <a href="#/login">
            <Button size="sm" variant="ghost">
              {t('auth.login')}
            </Button>
          </a>
          <a href="#/register">
            <Button size="sm" variant="primary">
              {t('auth.register')}
            </Button>
          </a>
        </div>
      )
    }
  />

  <main className={route === '/login' ? 'min-h-[calc(100svh-4rem)]' : 'mx-auto w-full max-w-7xl px-4 py-
10 sm:px-6'}>
    {initializing ? (
      <div className="flex justify-center py-20" role="status" aria-label={t('common.loading')}>
        <Spinner size="lg" />
      </div>
    ) : (
      <RouteView route={route} />
    )}
  </main>

  <footer className="border-t border-ink-200 bg-cream-50 py-6 text-center text-sm text-ink-400">
    <span className="inline-flex flex-wrap items-center justify-center gap-x-2 gap-y-1 px-4">
      <Logo size="sm" withWordmark={false} />
      <span>RakshaSafe • Women Safety & Disaster Management System</span>
      <span aria-hidden="true">•</span>
      <span>Developed by Vivek & Vaibhav</span>
    </span>
  </footer>
</div>
)
}

function RouteView({ route }: { route: AppRoute }) {
  switch (route) {
    case '/register':
      return <RegisterPage />
  }
}

```

```

case '/dashboard':
  return (
    <RequireAuth>
      <DashboardPage />
    </RequireAuth>
  )
case '/sos':
  return (
    <RequireAuth>
      <SosPage />
    </RequireAuth>
  )
case '/profile':
  return (
    <RequireAuth>
      <ProfilePage />
    </RequireAuth>
  )
case '/contacts':
  return (
    <RequireAuth>
      <EmergencyContactsPage />
    </RequireAuth>
  )
case '/resources':
  return (
    <RequireAuth>
      <ResourcesPage />
    </RequireAuth>
  )
case '/notifications':
  return (
    <RequireAuth>
      <NotificationsPage />
    </RequireAuth>
  )
case '/responder':
  return (
    <RequireResponder>
      <ResponderDashboardPage />
    </RequireResponder>
  )
case '/admin/reports':
  return (
    <RequireAdmin>
      <ReportsPage />
    </RequireAdmin>
  )
case '/report-unsafe':
  return (
    <RequireAuth>
      <UnsafeReportsPage />
    </RequireAuth>
  )
case '/admin/unsafe-reports':
  return (
    <RequireAdmin>
      <AdminUnsafeReportsPage />
    </RequireAdmin>
  )
case '/incident-detail':
  return (
    <RequireAuth>
      <IncidentDetailPage />
    </RequireAuth>
  )
case '/admin/facilities':
  return (
    <RequireAdmin>
      <AdminFacilitiesPage />
    </RequireAdmin>
  )
case '/admin/teams':
  return (
    <RequireAdmin>
      <AdminTeamsPage />
    </RequireAdmin>
  )

```

```

    )
    case '/admin/incidents':
      return (
        <RequireAdmin>
        <AdminIncidentsPage />
        </RequireAdmin>
      )
    case '/admin/incident-detail':
      return (
        <RequireAdmin>
        <AdminIncidentDetailPage />
        </RequireAdmin>
      )
    case '/admin/dashboard':
      return (
        <RequireAdmin>
        <AdminDashboardPage />
        </RequireAdmin>
      )
    case '/admin/users':
      return (
        <RequireAdmin>
        <AdminUsersPage />
        </RequireAdmin>
      )
    case '/admin/user-detail':
      return (
        <RequireAdmin>
        <AdminUserDetailPage />
        </RequireAdmin>
      )
    case '/admin/emergency-contacts':
      return (
        <RequireAdmin>
        <AdminEmergencyContactsPage />
        </RequireAdmin>
      )
    case '/login':
    default:
      return <LoginPage />
  }
}

function App() {
  return (
    <ThemeProvider>
    <I18nProvider>
    <ToastProvider>
    <AuthProvider>
    <Shell />
    </AuthProvider>
    </ToastProvider>
    </I18nProvider>
    </ThemeProvider>
  )
}

export default App

```

Listing 4.38 — frontend/src/lib/api.ts

Typed fetch wrapper with JWT handling and error mapping.

```

const API_BASE = import.meta.env.VITE_API_BASE_URL || '/api'
const TOKEN_KEY = 'rakshasaFe.token'

export class ApiError extends Error {
  status: number
  details?: unknown

  constructor(status: number, message: string, details?: unknown) {
    super(message)
    this.name = 'ApiError'
    this.status = status
    this.details = details
  }
}

```

```

    }
}

export function getStoredToken(): string | null {
  try {
    return window.localStorage.getItem(TOKEN_KEY) ?? window.sessionStorage.getItem(TOKEN_KEY)
  } catch {
    return null
  }
}

export function setStoredToken(token: string, persistent = true): void {
  try {
    if (persistent) {
      window.localStorage.setItem(TOKEN_KEY, token)
      window.sessionStorage.removeItem(TOKEN_KEY)
    } else {
      window.sessionStorage.setItem(TOKEN_KEY, token)
      window.localStorage.removeItem(TOKEN_KEY)
    }
  } catch {
    /* storage unavailable – session lasts for this tab only */
  }
}

export function clearStoredToken(): void {
  try {
    window.localStorage.removeItem(TOKEN_KEY)
  } catch {
    /* ignore */
  }
  try {
    window.sessionStorage.removeItem(TOKEN_KEY)
  } catch {
    /* ignore */
  }
}

interface ApiOptions {
  method?: string
  body?: unknown
  token?: string | null
  signal?: AbortSignal
}

interface ApiEnvelope<T> {
  success: boolean
  data?: T
  error?: string
  details?: unknown
}

type UnauthorizedHandler = () => void

let unauthorizedHandler: UnauthorizedHandler | null = null

/**
 * Register a callback invoked when an authenticated request receives 401
 * (expired/revoked token). AuthProvider wires this to sign the user out so a
 * stale session can never linger. Requests sent without a token (login,
 * register, logout) never trigger it.
 */
export function onUnauthorized(handler: UnauthorizedHandler | null): void {
  unauthorizedHandler = handler
}

/** Typed fetch wrapper for the RakshaSafe REST API. Throws ApiError on failure. */
export async function api<T>(path: string, options: ApiOptions = {}): Promise<T> {
  const token = options.token === undefined ? getStoredToken() : options.token
  let res: Response
  try {
    res = await fetch(`${API_BASE}${path}`, {
      method: options.method ?? 'GET',

```

```

    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    ...(options.body !== undefined ? { body: JSON.stringify(options.body) } : {}),
    ...(options.signal ? { signal: options.signal } : {}),
  })
} catch (err) {
  // Re-throw cancellations untouched so every caller's AbortError guard
  // works; otherwise a stale aborted request would surface as a failure
  // and mask successfully loaded data with an error panel.
  if (err instanceof DOMException && err.name === 'AbortError') throw err
  throw new ApiError(0, 'Cannot reach the server. Check your connection and try again.')
}

let envelope: ApiEnvelope<T>
try {
  envelope = (await res.json()) as ApiEnvelope<T>
} catch {
  throw new ApiError(res.status, 'Unexpected server response.')
}

if (!res.ok || envelope.success !== true || envelope.data === undefined) {
  if (res.status === 401 && token) unauthorizedHandler?.()
  throw new ApiError(
    res.status,
    envelope.error ?? `Request failed (${res.status}).`,
    envelope.details,
  )
}
return envelope.data
}

```

Listing 4.39 — frontend/src/pages/LoginPage.tsx

Sign-in screen with role-based landing.

```

import { useState, type FormEvent } from 'react'
import { useAuth } from '../lib/auth-context'
import { useI18n, type DictKey } from '../lib/i18n'
import { navigateTo } from '../lib/hash-route'
import { AuthBrand } from '../components/auth/AuthBrand'
import { Alert } from '../components/ui/Alert'
import { Button } from '../components/ui/Button'
import { Card, CardBody } from '../components/ui/Card'
import { Form } from '../components/ui/Form'
import { Input } from '../components/ui/Input'
import { ShieldIcon, MapPinIcon, UsersIcon, BellIcon, EyeIcon, EyeOffIcon, LockIcon, MailIcon } from
'../components/ui/icons'

const EMAIL_REGEX = /^[^@]+@^[^@]+\.[^@]+$/
const INDIAN_PHONE_REGEX = /^(+91[\s-])?[6-9]\d{9}$/

function normalizePhone(phone: string): string {
  const trimmed = phone.trim()
  if (!trimmed) return ''
  const normalized = trimmed.replace(/^\+91[\s-]?/, '').replace(/[\s-]/g, '')
  return normalized
}

/**
 * The backend accepts any non-empty identifier (email or phone) and returns
 * "Invalid credentials" for unknown accounts. Frontend validation must not
 * be stricter than that, otherwise real accounts (e.g. admin phones that
 * are not Indian-mobile-shaped) can never even submit the form.
 */
function validateIdentifier(identifier: string): string | null {
  if (!identifier.trim()) return 'validation.required'
  return null
}

/**
 * Mirror the backend lookup without corrupting the identifier: emails are
 * lowercased, Indian numbers are stripped to 10 digits (the registration

```

```

* convention), and anything else is sent verbatim so the server can match
* exactly what is stored.
*/
function normalizeIdentifier(identifier: string): string {
  const trimmed = identifier.trim()
  const lower = trimmed.toLowerCase()
  if (EMAIL_REGEX.test(lower)) return lower
  const normalized = normalizePhone(trimmed)
  if (INDIAN_PHONE_REGEX.test(trimmed) && normalized.length === 10) return normalized
  return trimmed
}

function validatePassword(password: string): string | null {
  if (!password) return 'validation.required'
  return null
}

interface FeatureItem {
  icon: React.ReactNode
  title: DictKey
  description: DictKey
}

const features: FeatureItem[] = [
  { icon: <ShieldIcon className="size-5" />, title: 'login.hero.f1t', description: 'login.hero.f1d' },
  { icon: <MapPinIcon className="size-5" />, title: 'login.hero.f2t', description: 'login.hero.f2d' },
  { icon: <UsersIcon className="size-5" />, title: 'login.hero.f3t', description: 'login.hero.f3d' },
  { icon: <BellIcon className="size-5" />, title: 'login.hero.f4t', description: 'login.hero.f4d' },
]

export function LoginPage() {
  const { login, busy, error, clearError } = useAuth()
  const { t } = useI18n()
  const [identifier, setIdentifier] = useState('')
  const [password, setPassword] = useState('')
  const [showPassword, setShowPassword] = useState(false)
  const [fieldErrors, setFieldErrors] = useState<Record<string, string>>({})

  function validateForm(): boolean {
    const errors: Record<string, string> = {}
    const identifierError = validateIdentifier(identifier)
    if (identifierError) errors.identifier = identifierError
    const passwordError = validatePassword(password)
    if (passwordError) errors.password = passwordError
    setFieldErrors(errors)
    return Object.keys(errors).length === 0
  }

  async function onSubmit(e: FormEvent): Promise<void> {
    e.preventDefault()
    if (busy) return
    if (!validateForm()) return
    try {
      const user = await login(normalizeIdentifier(identifier), password)
      navigateTo(user.role === 'ADMIN' ? '/admin/dashboard' : user.role === 'RESPONDER' ? '/responder' :
'/dashboard')
    } catch {
      /* error is surfaced through context */
    }
  }

  return (
    <div className="min-h-svh w-full flex flex-col overflow-hidden bg-cream-50 dark:bg-ink-950">
      /* Minimal Header */
      <header className="flex items-center justify-between px-4 py-3 border-b border-ink-100 dark:border-
white/10 bg-cream-50/80 dark:bg-ink-950/80 backdrop-blur-sm">
        <div className="flex items-center gap-2">
          <AuthBrand size="md" />
        </div>
        <div className="flex items-center gap-2">
          <a href="#/register" className="text-sm font-medium text-ink-700 hover:text-gold-600 dark:text-ink-
200 dark:hover:text-gold-400 transition-colors">
            {t('auth.register')}
          </a>
        </div>
      </header>
    </div>
  )
}

```

```

    </div>
  </header>

  { /* Main Content */ }
  <main className="flex-1 w-full flex flex-col lg:flex-row overflow-hidden">
    { /* Left: Hero/Branding Section */ }
    <section className="hidden lg:flex lg:w-1/2 flex-col items-center justify-center px-8 py-12 bg-gradient-to-br from-sky-50 via-cream-50 to-gold-50 dark:from-sky-950/30 dark:via-ink-900/50 dark:to-gold-950/30 overflow-hidden relative">
      { /* Decorative background blobs */ }
      <div className="pointer-events-none absolute -top-20 -left-20 size-72 rounded-full bg-sky-300/20 blur-3xl dark:bg-sky-500/10" />
      <div className="pointer-events-none absolute -bottom-20 -right-20 size-72 rounded-full bg-gold-300/20 blur-3xl dark:bg-gold-500/10" />

      <div className="relative z-10 flex w-full max-w-xl flex-col items-center gap-8 text-center">
        { /* Logo */ }
        <div className="w-full max-w-[28rem] rounded-2xl bg-white/80 p-6 shadow-xl shadow-ink-900/5 ring-1 ring-ink-900/5 dark:bg-white/5 dark:ring-white/10 backdrop-blur-sm">
          <AuthBrand size="hero" />
        </div>

        { /* Title & Tagline */ }
        <div className="space-y-3">
          <p className="text-3xl lg:text-4xl xl:text-5xl font-extrabold tracking-tight text-ink-950 dark:text-white">
            Raksha<span className="text-gold-500 dark:text-gold-300">Safe</span>
          </p>
          <p className="text-xs font-bold uppercase tracking-[0.3em] text-sky-700 dark:text-sky-300">
            {t('login.hero.badge')}
          </p>
        </div>

        { /* Main Heading */ }
        <div className="space-y-2 text-left max-w-xl">
          <h1 className="text-3xl lg:text-4xl font-extrabold tracking-tight text-ink-950 dark:text-white leading-tight">
            {t('login.hero.titleA')}
          </h1>
          <h1 className="text-3xl lg:text-4xl font-extrabold tracking-tight text-ink-950 dark:text-white leading-tight">
            {t('login.hero.titleB')}
          </h1>
          <p className="text-lg text-ink-600 dark:text-ink-300 leading-relaxed">
            {t('login.hero.subtitle')}
          </p>
        </div>

        { /* Feature Cards */ }
        <ul className="w-full space-y-3 pt-2" role="list" aria-label={t('login.hero.badge')}>
          {features.map((feature, index) => (
            <li key={index} className="flex items-start gap-3 text-left group">
              <span className="inline-flex size-11 shrink-0 items-center justify-center rounded-xl bg-white/70 text-sky-700 shadow-sm ring-1 ring-sky-200/60 dark:bg-white/10 dark:text-sky-300 dark:ring-white/10 transition-all group-hover:bg-gold-100 dark:group-hover:bg-gold-900/30 group-hover:ring-gold-300/50">
                {feature.icon}
              </span>
              <span className="min-w-0 pt-1">
                <span className="block text-sm font-semibold text-ink-900 dark:text-white">{t(feature.title)}</span>
                <span className="block text-xs leading-relaxed text-ink-500 dark:text-ink-400">{t(feature.description)}</span>
              </span>
            </li>
          ))}
        </ul>
      </div>
    </section>

    { /* Right: Login Form Section */ }
    <section className="flex w-full lg:w-1/2 items-center justify-center px-4 py-10 sm:px-8 overflow-y-auto">
      <div className="w-full max-w-md">
        { /* Mobile/Tablet: Compact Brand Header */ }

```

```

<div className="mb-8 lg:hidden flex flex-col items-center gap-2 text-center">
  <AuthBrand size="md" />
  <div className="space-y-1">
    <p className="text-xl font-extrabold tracking-tight text-ink-950 dark:text-white">
      Raksha<span className="text-gold-500 dark:text-gold-300">Safe</span>
    </p>
    <p className="text-[10px] font-bold uppercase tracking-[0.25em] text-sky-700 dark:text-sky-
300">
      {t('login.hero.badge')}
    </p>
  </div>
</div>

{/* Login Card */}
<Card className="shadow-xl shadow-ink-900/10 ring-1 ring-ink-900/5 dark:ring-white/10">
  <CardBody className="p-6 sm:p-8">
    {/* Logo */}
    <div className="mb-6 flex justify-center">
      <div className="w-fit rounded-xl bg-white p-2.5 shadow-sm ring-1 ring-ink-900/5 dark:bg-
white/95 dark:ring-white/20">
        <AuthBrand size="sm" />
      </div>
    </div>

    {/* Title & Subtitle */}
    <div className="mb-6 text-center">
      <h2 className="text-2xl font-extrabold tracking-tight text-ink-950 dark:text-white">
        {t('login.title')}
      </h2>
      <p className="mt-1.5 text-sm text-ink-500 dark:text-ink-400">
        {t('login.subtitle')}
      </p>
    </div>

    {/* Form */}
    <Form onSubmit={{(e: FormEvent) => void onSubmit(e)}}
      {error && (
        <Alert variant="danger" title={t('login.errorTitle')} onClose={clearError} className="mb-
4">
          {error}
        </Alert>
      )}

    {/* Email/Phone Field */}
    <Input
      label={t('login.emailPhone')}
      name="identifier"
      type="text"
      autoComplete="username"
      placeholder={t('login.emailPhonePlaceholder')}
      requiredMark
      value={identifier}
      onChange={(e) => setIdentifier(e.target.value)}
      error={fieldErrors.identifier ? t(fieldErrors.identifier as DictKey) : undefined}
      leftIcon={<MailIcon className="size-4" />}
    />

    {/* Password Field */}
    <Input
      label={t('login.password')}
      name="password"
      type={showPassword ? 'text' : 'password'}
      autoComplete="current-password"
      placeholder={t('login.passwordPlaceholder')}
      requiredMark
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      error={fieldErrors.password ? t(fieldErrors.password as DictKey) : undefined}
      leftIcon={<LockIcon className="size-4" />}
      rightElement={
        <button
          type="button"
          onClick={() => setShowPassword((v) => !v)}
          aria-label={t(showPassword ? 'auth.hidePassword' : 'auth.showPassword')}
          aria-pressed={showPassword}

```

```

        className="p-1 text-ink-400 hover:text-ink-700 dark:hover:text-ink-200 transition-
colors"
        >
        {showPassword ? <EyeOffIcon className="size-4" /> : <EyeIcon className="size-4" />}
        </button>
    }
    />

    {/* Submit Button */}
    <Button type="submit" fullWidth loading={busy} disabled={busy} className="mt-2 h-11 text-
base font-semibold">
    {busy ? t('login.submitting') : t('login.submit')}
    </Button>

    {/* Register Link */}
    <div className="mt-6 border-t border-ink-100 pt-4 text-center dark:border-white/10">
    <p className="text-sm text-ink-500 dark:text-ink-400">
    {t('login.registerPrefix')}{t(' ')}
    <Button type="button" variant="ghost" size="sm" className="p-0 h-auto text-gold-700
hover:text-gold-800 dark:text-gold-300 dark:hover:text-gold-200 font-semibold" onClick={() =>
navigateTo('/register')}>
    {t('login.registerLink')}
    </Button>
    </p>
    </div>
    </Form>
    </CardBody>
    </Card>
    </div>
    </section>
    </main>

    {/* Footer */}
    <footer className="border-t border-ink-100 bg-cream-50 py-4 text-center text-xs text-ink-400
dark:border-white/10 dark:bg-ink-950">
    <p className="font-medium">RakshaSafe • Women Safety & Disaster Emergency Response</p>
    <p className="mt-0.5">Developed by Vivek & Vaibhav</p>
    </footer>
    </div>
    )
}

```

Listing 4.40 — frontend/src/components/risks/RiskPanel.tsx

AI risk display: score, level, factors, disclaimer.

```

import { formatDateTime, riskLevelVariant, type RiskAssessment } from '../../lib/risks'
import { Alert } from '../../ui/Alert'
import { Badge } from '../../ui/Badge'
import { Button } from '../../ui/Button'
import { Card, CardBody, CardHeader } from '../../ui/Card'
import { EmptyState } from '../../ui/EmptyState'
import { Spinner } from '../../ui/Spinner'

interface RiskPanelProps {
  assessments: RiskAssessment[]
  loading: boolean
  assessing: boolean
  assessError: string | null
  canAssess: boolean
  noLocation: boolean
  onAssess: () => void
  onDismissError: () => void
  /** Empty-state copy. Defaults suit the incident owner; the admin view passes its own wording. */
  emptyTitle?: string
  emptyDescription?: string
}

/**
 * Assistive AI risk display. Every value shown comes from the stored
 * backend assessment (the backend score is the single source of truth –
 * this component never recalculates a risk level).
 */
function factorLabel(factor: string): string {
  switch (factor) {

```

```

    case 'priorityBase':
      return 'Incident priority'
    case 'typeModifier':
      return 'Incident type'
    case 'verifiedReports':
      return 'Verified unsafe-area reports nearby'
    case 'unverifiedReports':
      return 'Unverified reports nearby'
    case 'activeIncidents':
      return 'Active incidents nearby'
    default:
      return factor
  }
}
}
export function RiskPanel({
  assessments,
  loading,
  assessing,
  assessError,
  canAssess,
  noLocation,
  onAssess,
  onDismissError,
  emptyTitle = 'Risk assessment has not been performed yet.',
  emptyDescription = 'Run an assistive risk assessment for this incident\u2019s location.',
}: RiskPanelProps) {
  const latest = assessments[0]

  return (
    <Card>
      <CardHeader>
        title="AI risk assessment"
        description="Assistive analysis from stored incident and area data."
        action={
          canAssess && !noLocation ? (
            <Button size="sm" variant="outline" onClick={onAssess} disabled={assessing || loading}>
              {assessing ? 'Assessing...' : 'Assess risk'}
            </Button>
          ) : undefined
        }
      </CardHeader>
      <CardBody>
        {loading && (
          <span className="flex items-center gap-2 text-sm text-ink-500">
            <Spinner size="sm" /> Loading assessments...
          </span>
        )}
        {!loading && noLocation && (
          <p className="text-sm text-ink-500">
            No stored coordinates for this incident, so no assessment can be produced.
          </p>
        )}
        {!loading && !noLocation && assessError && (
          <div className="space-y-3">
            <Alert variant="danger" title="Risk assessment temporarily unavailable" onClose={onDismissError}>
              {assessError}
            </Alert>
            {canAssess && (
              <Button size="sm" variant="primary" onClick={onAssess} disabled={assessing}>
                {assessing ? 'Assessing...' : 'Try Again'}
              </Button>
            )}
          </div>
        )}
        {assessing && (
          <span className="flex items-center gap-2 text-sm text-ink-500">
            <Spinner size="sm" /> Assessing incident risk...
          </span>
        )}
        {!loading && !noLocation && !assessError && assessments.length === 0 && (
          <EmptyState>
            title={emptyTitle}
            description={emptyDescription}
            action={
              canAssess ? (
                <Button size="sm" variant="primary" onClick={onAssess} disabled={assessing}>
                  {assessing ? 'Assessing...' : 'Assess risk'}
                </Button>
              ) : undefined
            }
          </EmptyState>
        )}
      </CardBody>
    </Card>
  )
}

```

```

        </Button>
      ) : undefined
    }
  }
  />
})
{!loading && latest && (
  <div className="space-y-3">
    <div className="flex flex-wrap items-center gap-3 rounded-xl border border-ink-200/70 bg-cream-50
p-4">
      <div>
        <p className="text-xs font-bold uppercase tracking-widest text-ink-400">Risk Score</p>
        <p className="text-4xl font-extrabold text-ink-900">
          {latest.riskScore}
          <span className="text-lg font-semibold text-ink-400"> / 100</span>
        </p>
      </div>
      <div>
        <p className="text-xs font-bold uppercase tracking-widest text-ink-400">Risk Level</p>
        <div className="mt-1">
          <Badge variant={riskLevelVariant(latest.riskLevel)} dot>
            {latest.riskLevel}
          </Badge>
        </div>
        <p className="mt-1 text-xs text-ink-400">
          Model: {latest.modelVersion} • Assessed: {formatDateTime(latest.assessedAt)}
        </p>
      </div>
    </div>
    {latest.inputFactors.length > 0 && (
      <div>
        <p className="text-xs font-bold uppercase tracking-widest text-ink-400">Risk factors</p>
        <ul className="mt-1 space-y-1 text-sm">
          {latest.inputFactors.map((f) => (
            <li key={f.factor} className="flex items-center justify-between gap-2 text-ink-600">
              <span>
                {factorLabel(f.factor)}:{' '}
                <span className="font-semibold text-ink-800">{f.value}</span>
              </span>
              <span className="text-xs text-ink-400">+{f.weight}</span>
            </li>
          ))}
        </ul>
        <p className="mt-2 text-xs leading-relaxed text-ink-500">
          Risk level is calculated from the incident priority, incident type and
          verified safety information available around the reported location.
        </p>
      </div>
    )}
    {assessments.length > 1 && (
      <div>
        <p className="text-xs font-bold uppercase tracking-widest text-ink-400">Previous</p>
        <ul className="mt-1 space-y-1 text-sm text-ink-500">
          {assessments.slice(1, 4).map((a) => (
            <li key={a.id}>
              {a.riskScore} ({a.riskLevel}) • {formatDateTime(a.assessedAt)}
            </li>
          ))}
        </ul>
      </div>
    )}
    <Alert variant="warning" title="Decision support only">
      This score is an assistive risk assessment based on available stored data and
      does not replace professional emergency judgement.
    </Alert>
  </div>
})
</CardBody>
</Card>
)
}

```

Listing 4.41 — frontend/src/components/location/LocationDetail.tsx

Address-first location block with Open in Maps.

```

import type { IncidentLocation } from '../../lib/incidents'
import { buildMapsUrl, describeStoredArea } from '../../lib/resources'

```

```

import { formatDateTime } from '../lib/incidents'
import { Skeleton } from '../ui/Skeleton'

interface LocationDetailProps {
  location: IncidentLocation
  /** Live-resolved address (enrichment). Stored address always wins. */
  liveAddress?: string | null
  addressLoading?: boolean
  /** Wording for device-captured locations (user view) vs reviewed ones (admin view). */
  captureNote?: string
}

/**
 * Stored incident location block: readable address first, real coordinates
 * for verification, accuracy/capture metadata, and an "Open in Maps" link
 * built from the actual coordinates only. Never invents address parts:
 * missing data degrades to "Address unavailable - coordinates available".
 */
export function LocationDetail({
  location,
  liveAddress = null,
  addressLoading = false,
  captureNote = 'Location captured from the user\u2019s device. Accuracy depends on the device/browser
location provider.',
}: LocationDetailProps) {
  const storedArea = describeStoredArea(location)
  const mapsUrl = buildMapsUrl(location.latitude, location.longitude)

  return (
    <div className="space-y-3">
      <dl className="grid gap-3 text-sm sm:grid-cols-2">
        <div className="sm:col-span-2">
          <dt className="font-semibold text-ink-500">Address</dt>
          <dd className="mt-0.5 text-ink-900">
            {storedArea ?? liveAddress ?? (addressLoading ? <Skeleton lines={1} /> : 'Address unavailable -
coordinates available')}
          </dd>
        </div>
        <div>
          <dt className="font-semibold text-ink-500">Latitude</dt>
          <dd className="mt-0.5 text-ink-900">{location.latitude.toFixed(6)}</dd>
        </div>
        <div>
          <dt className="font-semibold text-ink-500">Longitude</dt>
          <dd className="mt-0.5 text-ink-900">{location.longitude.toFixed(6)}</dd>
        </div>
        {location.accuracy !== undefined && (
          <div>
            <dt className="font-semibold text-ink-500">Accuracy</dt>
            <dd className="mt-0.5 text-ink-900">±{Math.round(location.accuracy)} m</dd>
          </div>
        )}
        {location.capturedAt && (
          <div>
            <dt className="font-semibold text-ink-500">Captured</dt>
            <dd className="mt-0.5 text-ink-900">{formatDateTime(location.capturedAt)}</dd>
          </div>
        )}
      </dl>
      {mapsUrl && (
        <a
          href={mapsUrl}
          target="_blank"
          rel="noreferrer"
          className="inline-flex h-9 items-center justify-center gap-1.5 rounded-xl border border-ink-300 bg-
white px-3 text-sm font-semibold text-ink-700 transition-colors duration-150 hover:border-gold-400 hover:bg-
gold-50 hover:text-gold-700"
        >
          Open in Maps
        </a>
      )}
      <p className="text-xs text-ink-400">{captureNote}</p>
    </div>
  )
}

```

Listing 4.42 — frontend/src/components/map/SafetyMap.tsx

Leaflet map with OpenStreetMap tiles and markers.

```
import { useEffect, useRef } from 'react'
import type { Map as LeafletMap } from 'leaflet'
import 'leaflet/dist/leaflet.css'
import { useI18n } from '../../lib/i18n'

export interface SafetyMapMarker {
  latitude: number
  longitude: number
  label: string
  kind: 'user' | 'police' | 'hospital' | 'fire' | 'ambulance' | 'other'
}

interface SafetyMapProps {
  center: { latitude: number; longitude: number }
  markers: SafetyMapMarker[]
}

const MARKER_COLORS: Record<SafetyMapMarker['kind'], string> = {
  user: '#d49d26',
  police: '#247bb8',
  hospital: '#dc2626',
  fire: '#ea580c',
  ambulance: '#0d9488',
  other: '#16a34a',
}

/**
 * Real map: user position + nearby facilities from actual coordinates only.
 * Leaflet + OpenStreetMap tiles (no API key). The library loads lazily so
 * the rest of the page never depends on it; records without coordinates
 * never receive markers.
 */
export function SafetyMap({ center, markers }: SafetyMapProps) {
  const { t } = useI18n()
  const containerRef = useRef<HTMLDivElement | null>(null)
  const userLabel = t('nearby.yourLocation')

  useEffect(() => {
    let cancelled = false
    let map: LeafletMap | null = null
    async function init(): Promise<void> {
      const leaflet = await import('leaflet')
      if (cancelled || !containerRef.current) return
      map = leaflet.map(containerRef.current, { scrollWheelZoom: false })
      map.setView([center.latitude, center.longitude], 14)
      leaflet
        .tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
          maxZoom: 19,
          attribution: t('map.attribution'),
        })
        .addTo(map)
      leaflet
        .circleMarker([center.latitude, center.longitude], {
          radius: 9,
          color: 'ffffff',
          weight: 2,
          fillColor: MARKER_COLORS.user,
          fillOpacity: 1,
        })
        .bindTooltip(userLabel)
        .addTo(map)
      for (const marker of markers) {
        if (!Number.isFinite(marker.latitude) || !Number.isFinite(marker.longitude)) continue
        leaflet
          .circleMarker([marker.latitude, marker.longitude], {
            radius: 7,
            color: 'ffffff',
            weight: 2,
            fillColor: MARKER_COLORS[marker.kind],
            fillOpacity: 1,
          })
      }
    }
    init()
  }, [center, markers, t])
}
```

```

    })
    .bindTooltip(marker.label)
    .addTo(map)
  }
}
void init()
return () => {
  cancelled = true
  map?.remove()
  map = null
}
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [center.latitude, center.longitude, markers])

return <div ref={containerRef} className="z-0 h-64 w-full rounded-2xl sm:h-80" role="img" aria-label="Map
of your location and nearby facilities" />
}

```

Listing 4.43 — frontend/src/lib/geolocation.ts

One-shot device location with explicit permission states.

```

export interface DeviceCoords {
  latitude: number
  longitude: number
  accuracy?: number
}

export type LocationOutcome =
  | { state: 'available'; coords: DeviceCoords }
  | { state: 'denied' }
  | { state: 'timeout' }
  | { state: 'unavailable'; message: string }
  | { state: 'unsupported' }

export function isGeolocationSupported(): boolean {
  return typeof navigator !== 'undefined' && 'geolocation' in navigator
}

function toOutcomeError(message: string): LocationOutcome {
  return { state: 'unavailable', message }
}

/**
 * Request the device/browser position once. Never invents coordinates:
 * every non-available outcome is reported explicitly so the UI can
 * communicate the actual state instead of assuming a false location.
 */
export function requestDeviceLocation(timeoutMs = 15000): Promise<LocationOutcome> {
  if (!isGeolocationSupported()) {
    return Promise.resolve({ state: 'unsupported' })
  }
  return new Promise((resolve) => {
    let settled = false
    const timer = window.setTimeout(() => {
      if (!settled) {
        settled = true
        resolve({ state: 'timeout' })
      }
    }, timeoutMs)

    navigator.geolocation.getCurrentPosition(
      (pos) => {
        if (settled) return
        settled = true
        window.clearTimeout(timer)
        const { latitude, longitude, accuracy } = pos.coords
        if (!Number.isFinite(latitude) || !Number.isFinite(longitude)) {
          resolve(toOutcomeError('The device returned an invalid position.'))
          return
        }
        resolve({
          state: 'available',
          coords: {

```

```

        latitude,
        longitude,
        ...(typeof accuracy === 'number' && Number.isFinite(accuracy) && accuracy >= 0
        ? { accuracy }
        : {}),
    },
  })
},
(err) => {
  if (settled) return
  settled = true
  window.clearTimeout(timer)
  if (err.code === err.PERMISSION_DENIED) {
    resolve({ state: 'denied' })
  } else if (err.code === err.TIMEOUT) {
    resolve({ state: 'timeout' })
  } else {
    resolve(toOutcomeError(err.message || 'Position unavailable.'))
  }
},
{ enableHighAccuracy: true, timeout: timeoutMs, maximumAge: 0 },
)
})
}

export function describeOutcome(outcome: LocationOutcome): string {
  switch (outcome.state) {
    case 'available':
      return 'Device location acquired.'
    case 'denied':
      return 'Location permission was denied. You can still submit without coordinates.'
    case 'timeout':
      return 'Locating timed out. You can retry or submit without coordinates.'
    case 'unavailable':
      return outcome.message
    case 'unsupported':
      return 'This browser or device does not support geolocation.'
  }
}

/**
 * Reverse geocode coordinates to a human-readable address using OpenStreetMap Nominatim.
 * Returns the address string or null if unavailable.
 */
export async function reverseGeocode(latitude: number, longitude: number): Promise<string | null> {
  try {
    const res = await fetch(
      `https://nominatim.openstreetmap.org/reverse?format=json&lat=${latitude}&lon=${longitude}&zoom=16&addressdetails=1`,
      {
        headers: { 'Accept': 'application/json' },
        signal: AbortSignal.timeout(8000),
      }
    )
    if (!res.ok) return null
    const data = await res.json()
    const addr = data.address
    if (!addr) return null
    const parts = [
      addr.suburb || addr.neighbourhood || addr.quarter || addr.village,
      addr.city || addr.town || addr.district || addr.county,
      addr.state,
    ].filter(Boolean)
    return parts.join(', ') || data.display_name || null
  } catch {
    return null
  }
}

```

4.8 Running the System

The three runtime processes use the ports and commands documented in the project README: the Vite development server on port 5173 (proxying /api to the backend and /ai to the AI service), the

Express REST API on port 5000 with its health endpoint at `/api/health`, and the FastAPI service on port 8000 with interactive docs at `/docs`. Each service reads its own local `.env` file (see the `.env.example` templates); no secret is embedded in the code listed above. Production builds are produced with `tsc` and `vite build` (frontend) and `tsc` (backend), exactly as verified during development.

4.9 Summary

This chapter has presented 43 complete source listings totalling 3472 lines of implementation code across the backend, the AI service and the frontend. Together with the analysis and design of Chapter 3, they document a working, tested Women Safety and Disaster Emergency Response System: authenticated users report geolocated SOS incidents, administrators manage the full response workflow, responders handle their own assignments, and a deterministic AI service provides explainable, reproducible risk assessments as decision support.